

Anomaly-Aware ForensiBlock: Explainable Behavioral Monitoring for Digital Evidence Access

Asma Jodeiri Akbarfam
Nishi Kanta Paul
Shereen Ismail

Abstract—Blockchain digital-forensic models incorporate provenance as well as role-based access control (RBAC) based on stages of the process being performed. However, static authorization does not enable detection of anomalies in the behavior of insiders authorized to access a system from a technical perspective. We propose the Anomaly-Aware ForensiBlock system as an extension to RBAC-SA model and add real-time behavioral anomaly score, risk adaptive response, asynchronous explainability, and verifiable auditing without breaking the current provenance path. Synchronous path computes behavioral and forensic context features using rolling statistics prior to the event occurrence and determines a risk level and corresponding response for each allowed request. The background process records the anomaly on-chain prior to the computation of the explanation, computes the TreeSHAP explanation, encrypts and saves the explanation off-chain, and links the integrity hash and save location of the report with another transaction on the Anomaly Ledger. This assessment differentiates a generalization on an external behavior benchmark from a verification of the prototype integration. XGBoost obtained an AUPRC of 0.257 and AUROC of 0.953 in 30 re-splits of CERT r4.2 (Layer A, generalization), $\sim 80\times$ the AUPRC of no skill given the class imbalance. With respect to a full schema corpus simulation (Layer B, integration validation), Random Forest performed at AUPRC of 0.745 and F_1 -score of 0.778, correctly identifying Critical risks in four out of five injected scenarios per test event, while off-hours access continued to prove challenging to differentiate from actual access patterns. The anomaly scoring overhead was measured to be on average 9.37 ms ($P99 = 10.37$ ms) for the run with integrated processing. All 25 alerts were successfully put into queues, logged, explained, and linked. All 65 audited transactions were processed without failure or linkage issues. This performance data comes from a partial integration prototype in which the access path is in-process, and the Anomaly Ledger is the contract called for each request.

Index Terms—Behavioral anomaly detection, blockchain, digital forensics, explainable artificial intelligence, insider threat, role-based access control, smart contracts, tamper-evident audit, forensic provenance.

I. INTRODUCTION

The probative worth of computer evidence relies entirely upon its handling's integrity and trackability. The Chain of Custody must demonstrate beyond doubt that there was no alteration, substitution, or access of the evidence in any manner that did not have legal authority from the time of collection until the time of its introduction into the courtroom, where even one break in the chain may render evidence that could be conclusive inadmissible [1], [2]. Traditional custody records, which are centrally managed, fall short as we move away from centralized forensics [3].

This problem is solved by using the method of managing evidence with the help of blockchain technology where the handling of evidence is recorded in a tamper-evident cryptographic structure that supports decentralized verification and an audit path no single party can alter in secret [4], [5]. A common example is ForensiBlock [6] that represents a permissioned blockchain with four smart contracts, role-based access control and stage awareness (RBAC-SA), Merkle-root integrity verification and encryption off-chain storage which guarantees that the access to the evidence by any authorized user during a particular stage of investigation is possible.

Such frameworks give one clear security answer — *is this user authorized to have access to this evidence?* Only in cases where the role of the user and the phase of the investigation meet a specific permission policy can there be access. This works as a control for external attackers. But these methods fail to address the equally significant issue of *whether the authenticated user is behaving in a way that is consistent with legitimate investigation processes*. **RBAC-SA checks identity and entitlement statically**; it does not understand what is considered typical behavior in accessing resources, nor does it understand when an entitled user does not exhibit such behavior.

Insider abuse is known to be one of the hardest security threats to detect in terms of both expense and difficulty since the person committing it already has credentials that are legitimate [7]–[9], and the implications of the attack are serious from a forensic perspective. All the RBAC-SA tests will pass for an investigator who conducts themselves in accessing sensitive pieces of evidence during off hours, downloading large amounts of case data within a very short time frame, conducting an investigation at a stage which goes against past history, or succeeding after many failed attempts at authentication.

Closing the gap requires meeting three criteria that cannot be met by a mere anomaly detector. First, the behavioral assessment needs to be *explainable*. An isolated machine learning flag is not enough for forensic review and auditing because it does not allow the human auditor to understand why certain access was flagged as suspicious, what exactly about it made the behavior suspicious and so on [10]–[12]. Second, it needs to be *tamper-evident*. If there is any automation involved in the decision who will get access to the evidence, it also has to be tamper-evident. And it should also be *non-disruptive*, meaning all the guarantees provided by

ForensiBlock should be preserved, and the access-critical path should not be blocked by time-consuming analyses.

To address these challenges at once, we present **Anomaly-Aware ForensiBlock** that integrates a behavioral anomaly detection layer right after the RBAC-SA permission checking step, while preserving all the original components of ForensiBlock unchanged. The lightweight synchronous component calculates a score for each successful access attempt and responds according to a risk-based strategy, and then **explains the flagged cases using SHapley Additive exPlanations (SHAP) [13]** and logs them in a tamper-evident ledger using an asynchronous component.

Contributions of this research can be summarized as follows:

- **Forensic provenance system access control layer with behavioral anomaly awareness.** An enhancement to the existing ForensiBlock system with respect to the non-intrusive integration of the behavioral anomaly detection phase in the technically authorized access requests without changing any internal structure of the ForensiBlock system. The feature extraction technique utilizes only rolling statistics before the event, which means that none of the features is extracted based on the outcome of the currently processed request.
- **Architecture for asynchronous explainability of access decisions with high stakes involved.** The explanation will not interfere with the access because the SHAP-based feature attributions for each flagged access are calculated by a separate background worker and not in the access decision path. Each flagged access will have an explanation that gives transparency to the audit through human-readable descriptions of behavioral attributes behind.
- **Tamper-evident audit trail of AI decisions while maintaining privacy.** This is done by employing a special smart contract called the Anomaly Ledger that records decisions on chain with a two-transaction approach. The anomaly event is committed by the background worker before producing any explanations, and after those explanations have been generated, they are cryptographically tied to them. Placing the write for the detection ahead of the generation of an explanation ensures that the audit trail lives on even in the case where the creation of the explanation is deferred or fails entirely. Hence, the ledger provides proof of the integrity of decisions made, but not completeness of the anomalies recorded, since the transfer of detections into the worker is an in-memory queue.

The evaluation is organized in two layers. Layer A evaluates how well the anomaly detector generalizes to the behavior of an authorized user, **using release r4.2 of the CERT Insider Threat Test Dataset [14] as an external benchmark**. Layer B evaluates the complete access–explanation–ledger pipeline on a simulation of the access logs used in ForensiBlock. The detection quality, latency of the synchronous path, overhead of on-chain operations, and the scalability with respect to concurrent loads are evaluated.

The rest of this paper is organized as follows. Background information on blockchain forensic provenance, RBAC-SA and the resulting insider threat surface is provided in Section II, followed by related work in Section III. The architecture of Anomaly-aware ForensiBlock is described in Section IV, with the reference implementation in Section V. The experiment protocol is described in Section VI, followed by algorithms and complexity analysis in Section VII. Results are presented in Section VIII, and Section IX concludes the paper and states its limitations.

II. BACKGROUND

Four bodies of technique meet in the proposed extension: the blockchain provenance and stage-aware access control that ForensiBlock builds on, the insider threat surface that static authorization leaves open, behavioral anomaly detection, and explainable AI. This section covers each in turn.

A. Digital Forensic Provenance on Blockchain

Digital forensics provenance is the capability to demonstrate, at some future time, exactly how a particular evidence was created, retained and accessed. This problem is particularly well-suited to solve with the help of a distributed ledger because it is append-only and cannot be modified unilaterally by any single participant [1], [4], [5]. However, while a permissioned blockchain limits access to authorized nodes, this is acceptable in the restricted and accountable setting of forensic investigations, while still offering tamper evidence and decentralized verification.

This is provided by the use of four smart contracts in ForensiBlock system [6]: a *Tokenized* contract issuing unique token for each evidence object at upload time and giving derived objects tokens that are hashes of tokens of their parents thus providing on-chain evidence of their provenance; a *Case* contract including case data, its current stage of investigation, and the list of its tokens; an *Access Control* contract applying permission policy to all requests; and a *Provenance* contract getting case data and validating it using distributed Merkle root.

These kinds of systems segregate on-chain and off-chain storage of data. On-chain storage of evidence files is infeasible since it entails exposing confidential information and extremely high costs. Therefore, encrypted evidence and their provenance data are stored off-chain, while only metadata of the documents, tokens, and integrity hashes are stored on-chain. The integrity verification is done through the recalculation of an off-chain artifact hash and its comparison with an on-chain hash.

B. Stage-Aware Role-Based Access Control

Access to the forensic evidence in ForensiBlock system is **controlled by RBAC-SA [6]. Classical role-based access control (RBAC) grants permissions to a user according to role membership rather than personal identity**; RBAC-SA recognizes the fact that the same role can need different

permissions at different stages of the case, as one action may be suitable for one case stage and unsuitable for another.

The evidence access request at time t is represented by $r_t = \langle u_t, c_t, e_t, a_t, g_t \rangle$, where u_t , c_t , e_t , a_t , and g_t stand for user, case, evidence item, access permission and case investigation stage. RBAC-SA makes a decision about whether a policy predicate is allowed or denied for the combination of user role and current phase,

$$\text{RBAC-SA}(r_t) \in \{\text{allow}, \text{deny}\}, \quad (1)$$

allowing access only if the particular role–phase combination is a match in the predefined permission matrix. It performs a constant-time test against a permission matrix and is decisive in order to prevent outsiders from gaining access. However, the important thing to note is that it is an inherent *static* relationship between *identity* and *privilege* and is unable to capture any knowledge or representation of user behavior.

C. Threat Surface for Insiders

The insider threat surface that constitutes the scope of this research effort is based on the aforementioned restriction. By definition, an insider threat involves a user with legitimate credentials who abuses them in an illegitimate manner regardless of the corresponding permission [7]–[9]. These types of attacks are regarded as particularly dangerous and hard to catch because of the fact that the actor is operating within the trust boundary, which means that there is no authorization violation [15], [16].

This problem can be formulated explicitly. RBAC-SA separates the space $\mathcal{R}$ of all access requests into two categories: $\mathcal{R}_{\text{allow}}$, which includes the permissible requests, and $\mathcal{R}_{\text{deny}}$, which includes the forbidden ones, as shown in Equation (1). Among the permissible requests there are behaviorally legitimate ones; we denote their set as $\mathcal{R}_{\text{legit}}$. A static policy implicitly assumes that $\mathcal{R}_{\text{allow}} \subseteq \mathcal{R}_{\text{legit}}$, that is, that everything it authorizes is also behaviorally appropriate. In practice this assumption does not hold, and it leaves a nonempty set

$$\mathcal{R}_{\text{anom}} = \mathcal{R}_{\text{allow}} \setminus \mathcal{R}_{\text{legit}}, \quad (2)$$

where each element is authorized yet anomalous. RBAC-SA cannot tell whether a request is legitimate or belongs to $\mathcal{R}_{\text{anom}}$ because, by its nature, it maps all elements of $\mathcal{R}_{\text{anom}}$ to *allow*. This nonempty set is the insider threat blind spot and the purpose of the presented enhancement is to identify it.

This limitation of RBAC-SA is clearly demonstrated in Figure 1, which shows that each request having a proper role–stage pair will be allowed without being analyzed for any suspicious actions. Such behavior is accepted according to the policy, since it is not inspected by the policy. For common forensic scenarios, as depicted in Table I, RBAC-SA allows the requested user, who is supposed to have the correct role for that particular scenario, access. Nonetheless, in each scenario, there is always a known sign that indicates the potential tampering, exfiltration, or credential theft of the evidence. What is abnormal is not *who* is doing *what* they have been authorized to do, but *how* they do it.

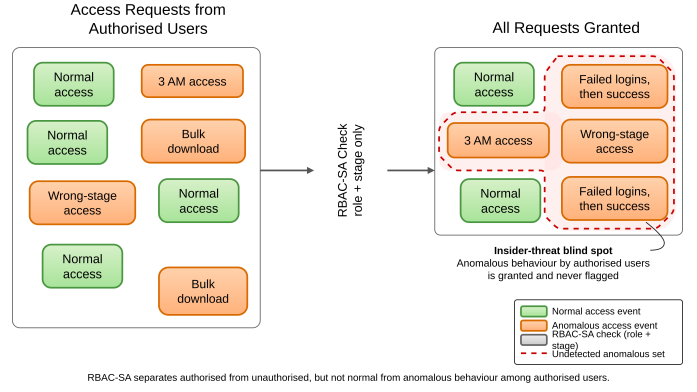


Fig. 1. The insider-threat blind spot in RBAC-SA. Access requests from authorized users contain both normal and anomalous behavioral events. As RBAC-SA only checks the role–stage permission (Equation (1)), all requests with a valid role and stage pass through, hence anomalous behavior by an authorized user, such as off-hours or bulk-download activity (see Table I for the full list), goes undetected. This unobserved subset is $\mathcal{R}_{\text{anom}}$ in Equation (2).

TABLE I
INSIDER-THREAT SCENARIOS THAT RBAC-SA CANNOT DETECT. IN ALL CASES, THE ROLE–STAGE PERMISSION IS VALID AND ACCESS IS GIVEN.

Observed behavior	Undetected risk
Evidence obtained at an irregular hour	Off-hours access anomaly
Large number of files downloaded in a short time interval	High-frequency access; possible exfiltration
Case stage access inconsistent with user history	Stage-pattern deviation
Many failed logins then a successful login	Credential-abuse signal
Abnormal frequency of provenance extraction	Unusual reconnaissance of evidence lineage

D. Behavioral Anomaly Detection

Deciding whether an authorized request belongs to $\mathcal{R}_{\text{anom}}$ can be posed as a behavioral anomaly detection problem. An access event is encoded as a feature vector, which captures the behavioral and forensic context of the access request: access time, access rate, access session, deviation from the past role–stage sequence of the user, and forensic-specific features. A classifier transforms the vector into a real-valued risk score.

Two classes of methods can be used, depending on whether forensic application has labeled examples of insider behavior. Methods like Isolation Forest [17] identify the anomaly as an outlier, which is clearly separated from most of the data and do not use any labeling information. Methods like Random Forest [18] and XGBoost [19] are labeled-data-dependent and make more accurate decisions where labeling exists. Tree-based ensembles have four key strengths as applied to this particular task: they are naturally capable of working with tabular data of mixed types, they have built-in class imbalance protection through use of sample weights, they are inexpensive to run without any special equipment and they have exact explainability, which is described further below.

Real-time requirements imply behavioral features which can

be calculated without looking through the entire history of each user. Welford’s incremental algorithm [20] calculates mean, variance and number of elements in a single pass and constant space per each statistic. Moreover, it has numerically stable updates that do not store any individual observations.

However, there are many more instances of normal behavior than there are of anomalies, so accuracy does not adequately measure quality, as the model could always return nothing and would still be quite accurate. **For our assessment, we employ precision, recall, the F_1 score, the area under the receiver-operating-characteristic curve (AUROC), and the area under the precision–recall curve (AUPRC). We also consider the false positive rate (FPR), a vital operational metric because each false positive increases the load on the investigators.**

E. Explainable AI for Security Decisions

Not only must a risk score be provided, but the decision based on it must also come with a human-understandable explanation [10], [11], [21]. This is solved using XAI (Explainable AI), which traces back the prediction of the model to the features present in the input that led to this output [22]. **The SHAP methodology [13] considers the features of an observation as players and then distributes the prediction among them based on Shapley values [23], the only method of attribution that fulfills a set of axioms like local accuracy, such that the attributions add up to the model’s output:**

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^d \phi_j(\mathbf{x}), \quad (3)$$

where ϕ_0 is the expected model output over the background distribution and $\phi_j(\mathbf{x})$ is the contribution attributed to feature j for the observation $\mathbf{x}$. Features can then be ranked by the magnitude of ϕ_j and summarized concisely in natural language.

On average, it is impossible to calculate the Shapley values exactly because the cost of doing so is exponential to the number of input variables and therefore prohibitive in real-time scenarios. In the case of tree-based models, however, it is possible to calculate the Shapley values exactly using the TreeSHAP algorithm [24], which is of complexity $\mathcal{O}(TLD^2)$, where T is the number of trees in the ensemble, L is the maximum number of leaves in each tree, and D is the maximum depth.

With these foundations in place, Section III positions the proposed work against prior research that draws on them.

III. RELATED WORK

To contextualize the proposed framework, existing literature is organized across four key research domains: blockchain-based forensic provenance, insider threat detection, explainable AI for cybersecurity and digital forensics, and adaptive/explainable access control. Although these research avenues achieve significant advancements, they remain fundamentally disconnected in the literature. Notably, current state-of-the-art access control and anomaly detection systems lack a mechanism to generate cryptographic, tamper-evident proofs

for their XAI attributions, creating a critical vulnerability in forensic chain-of-custody pipelines.

A. Blockchain Digital Forensics and Provenance

Blockchain technologies have been used in digital forensics for improving the integrity and traceability of records related to the handling of the evidence. Bonomi *et al.* [1] suggested an architecture called B-CoC for blockchain chain of custody based on permissioned Ethereum, which separates off-chain storage of the evidence from the on-chain custody operations. Lavín and Llanos [2] compared blockchain chain of custody solutions in terms of adherence to ISO/IEC 27037 and identified persistent issues related to forensic standardization and admissibility. At the architectural level, Jodeiri Akbarfam *et al.* [4] have formalized the requirements of blockchain provenance in single-entity, collaborative and multi-chain scenarios, specifying investigation stage-based modeling, role-based access, privacy, interoperability and evidence integrity as the primary dimensions of design.

ForensiBlock [6] (Section II-A) is another blockchain-based provenance architecture that uses role- and investigation-stage-aware access control. However, the decision to grant authorization in the case of ForensiBlock is made using the static role–staging predicate without taking into account the abnormality of the authorized request.

The latest research has expanded the scope of application of such controls: Xiao *et al.* [25] combine smart contract technology, token-based access control and evidence retrieval in the context of **the Industrial Internet of Things (IIoT)**. Kumar and Ali [26] develop authentication in forensics of IoT, Nath *et al.* [27] study existing chain of custody requirements, Loffi *et al.* [28] integrate blockchain with self-sovereign identity, and existing reviews [5], [29] cover the relevant literature. Although all these approaches provide significant improvements in terms of provenance, integrity, authentication and enforcement of policies, they do not consider abnormality of behavior of a policy-satisfied user as a reason to deny their access. In other words, an authorized user who behaves abnormally, excessively or inconsistently with the circumstances of the investigation cannot be distinguished from other users.

B. Insider Threat Detection and User Behavior Analytics

This approach covers methods for detecting behavioral anomalies of authorized users using organizational logs. One persistent issue is that of unrepresentative data; Lindauer *et al.* [14] created a synthetic dataset for the CERT Insider Threat Project since enterprise logs are not available for privacy, business, and national security reasons. This issue is prevalent in many of the studies that come later and also in our current study.

Kim *et al.* [7] utilized the CERT dataset to demonstrate the conversion of the heterogeneous data logs into behavior features, followed by the use of one-class methods in the case of extreme class imbalance. Le and Zincir-Heywood [8] established the importance of temporal aggregation using the sessions, days, and weeks, and also presented metrics for

the analyst, such as per user detection rate and detection delay. Le *et al.* [15] also provide measurements for the connection between the time grain, accuracy, false positives and timeliness. The surveys [9], [16] demonstrate that the development of approaches goes from statistical and one-class classification approaches to deep, hybrid and representation learning approaches involving log-texts analysis [30].

The more advanced systems go beyond the mere classifying phase. E-Watcher [31] uses a combination of statistically driven and Local Outlier Factor approach on the behavioral profiles of users, and its CERT r4.2 evaluation considers the management of noise, imbalanced data, and the proportion of malicious activities in fitting the insider threat detectors. Qawasmeh and AlQahtani [32] use anomaly detection and dynamic user profiling to assign users into low, medium, and high-risk classes.

Collectively, the literature above indicates that modeling of the behavior of legitimate users is possible and that anomaly scores could help in developing a tailored response. Nonetheless, the models are built using generic activities of organizations and do not consider the forensic scenario described in Section II-D, i.e., investigative phase, evidence sensitivity, and behavior related to evidence handling. Furthermore, the judgment does not consider the forensic access control decision and the provenance trail of evidence handling.

C. XAI for Cybersecurity and Digital Forensics

The XAI approaches in Section II-E have been used mostly for intrusion detection in previous cybersecurity research. **Local interpretable model-agnostic explanations (LIME) [22] is one widely used approach.** Wang *et al.* [10] incorporated both local and global SHAP explanations in NSL-KDD-based intrusion detection. Mane and Rao [11] have used LIME, Contrastive Explanations, ProtoDash, and Boolean Rules with respect to separating the informational requirements of model developers from security operators. Subsequent research [12], [33], [34] explores the application, evaluation, transparency, and trustworthiness of post hoc explanation of network intrusion detection. Other literature [21], [35], [36] concurs that the field of explanation is increasingly becoming crucial within cybersecurity operations.

XAI is increasingly being considered for digital forensics too. Khalid *et al.* [37] apply Isolation Forests, autoencoders, k-means clustering, DBSCAN, and Gaussian Mixture Models on memory forensic data and explain their outputs via SHAP values. However, they focus on explanation of memory evidence about the malware, not the behavior of the legitimate user seeking access. Another concern is the quality of the explanation; Calzarossa *et al.* [38] evaluate explanation using prediction accuracy, simplicity, parsimony, and robustness, a sign that generating a SHAP vector does not imply stability or operational explanation by itself.

These papers make security and forensic models more inspectable, but still, the explanations have no relation to the access and provenance controls that govern a forensic case. They can be either provided to an analyst or saved as regular

analytical output. However, these logs are neither cryptographically connected to the access event nor safeguarded against future modifications or associated with a custody log. Hence, the issue raised here is not just about explaining the behavioral determination but rather ensuring that this explanation does not get lost from the accountability trail that supports the decision-making process.

D. Adaptive and Explainable Access Control

Machine learning techniques have also been examined to enhance access control by incorporating rules generated by algorithms. Nobi *et al.* [39] classify such studies on the basis of attribute extraction, role and policy mining, policy verification, and policy monitoring as well as machine-learning based access decisions. According to the authors, learning can be beneficial in cases when static policies do not capture dynamic user-resource associations, citing issues relating to small data sets and explainability and manageability of model-based decisions. Hasel Mehri *et al.* [40] go a step further and leverage the need for explainability in order to develop a model of explainable access control, which represents decisions through explanation graphs, and takes into account the explanation quality defined by relevance, complexity, consistency, and usefulness for stakeholders.

This body of literature is thus relevant both to adaptive decision-making and explanations for stakeholders, although the main topic here is policy management and explanation per se. Behavioral anomaly detection is not used in the forensic workflow, does not have investigation-specific features, and does not provide any form of tamper-evident audit functionality. This is a set of capabilities that depends on the access decision, behavioral context, explanation process and provenance layer being seen as an integral accountability workflow.

E. Summary and Positioning

From Table II, it is clear that these research efforts are increasingly overlapping, and recent literature connects several historical boundaries among them. Insider threat analysis is becoming more of a hybrid and real-time approach; unsupervised explainable AI directly integrates explainability into the field of digital forensics; and research into explainable access control defines the means by which decisions can be conveyed to various actors. Nonetheless, these capabilities remain distinct from each other.

Anomaly-Aware ForensiBlock lies in the intersection of these areas. It incorporates the stage and role awareness of the authorization and blockchain-based provenance functionality of ForensiBlock, performs the behavioral analysis on authorized forensic requests and the score generated by the analysis and maps the same to a risk-stratified response. The SHAP explanation is completely separated from the time-critical access chain. The audit process is designed to collect the metadata on the anomaly and cryptographically link the generated explanation to the event. The novelty of this work is

TABLE II
COMPARISON OF RELATED-WORK CATEGORIES AND ANOMALY-AWARE FORENSIBLOCK

Work Category	Blockchain-Enabled Provenance	Forensic Access Control	Runtime Anomaly Detection	Authorized-User Behavior Modeling	Explainable Decisions	Tamper-Evident Explanation Logging
Blockchain Forensics [1], [2], [4], [6], [25]–[28]	✓	✓				
Insider Threat Detection and User Behavior Analytics [7]–[9], [14], [15], [30]–[32]			✓*	✓		
XAI for Cybersecurity and Digital Forensics [10]–[12], [33], [34], [37], [38]			✓†		✓	
Adaptive and Explainable Access Control [39], [40]		✓‡			✓‡	
Anomaly-Aware Forensi-Block (Ours)	✓	✓	✓	✓	✓	✓

*Applied to generic enterprise activity or synthetic user profiles, rather than integrated forensic access events.

†Anomaly explanation shown but not for authorized forensic access or tamper-evident preservation.

‡Access-control support, general-purpose or conceptual, not an end-to-end forensic anomaly-response workflow.

not in building an anomaly detection, explanation, and custody system, but in integrating them in one process.

IV. METHODOLOGY

Anomaly-Aware ForensiBlock adds behavioral anomaly detection for users who are technically authorized to use ForensiBlock [6]. It is inserted directly after the RBAC-SA permission check, as is illustrated in Figure 3; permissioned blockchain, smart contracts, encrypted off-chain storage, and Merkle-root validation are left intact. We will first present the complete process of ForensiBlock (Section IV-A), followed by the newly added anomaly-aware part (Section IV-B).

A. ForensiBlock: Original Process

ForensiBlock is built on top of a permissioned blockchain that consists of three components: the permissioned blockchain itself, user nodes, and encrypted off-chain storage. Four smart contracts (Section II-A) are responsible for guiding a case through five steps, as shown in the original ForensiBlock workflow in Figure 3:

- 1) **Registration.** The chain is appended by the public key of the user, who thereby identifies themselves.
- 2) **Case creation.** Creation of the `TInitialUpload` transaction leads to the creation of the per-case Case contract and setting the permissions of the roles/stages via the Access Control contract; the case data is encrypted and stored off-chain.
- 3) **File upload.** Every `TFileUpload` transaction creates a new file token via the Tokenized contract; the content is encrypted and stored off-chain. Files derived from another file (`TAnalysis`) get tokens that are a hash of the parent tokens.
- 4) **Access request.** The Access Control contract checks the `TAccReq` transaction and grants access to the user or denies them according to their role and case stage. A

`TStage` transaction may move the case forward at any point and modify the permissions.

- 5) **Provenance recovery.** The Provenance contract accesses the off-chain storage to retrieve the case records and compares them against the on-chain Merkle root.

The suggested extension does not change anything about that process.

B. Anomaly-Aware ForensiBlock: The New Layer

The evidence access request is defined as $r_t = \langle u_t, c_t, e_t, a_t, g_t \rangle$, where u_t is the user, c_t is the case, e_t is the evidence, a_t is the action, and g_t is the investigation stage. The RBAC-SA check remains the same and returns allow/deny:

$$\text{RBAC-SA}(r_t) \in \{\text{allow}, \text{deny}\}. \quad (4)$$

In case of deny, ForensiBlock behaves in accordance with its native deny procedure. No modifications are done on the current ForensiBlock contract. If permitted, feature extraction, anomaly score calculation, risk classification, response choice, queue insertion for flagged events, and rolling state update processes are all performed in sync, returning the result after completion. When a flagged event is dequeued, as depicted in Figure 3, the background worker generates the explanation and writes to the Anomaly Ledger sequentially, outside the access-critical path.

1) *Feature extraction and anomaly score (off-chain, synchronous):* The system generates a behavioral feature vector

$$\mathbf{x}_t = \phi(r_t, H_u, H_c, H_e), \quad (5)$$

where H_u , H_c , H_e denote the historical access profiles of the user, case and the evidence item. A pre-trained machine learning model f_θ subsequently computes the risk score

$$s_t = f_\theta(\mathbf{x}_t). \quad (6)$$

These two operations are done in the application layer and hence *off-chain* and do not contribute any blockchain delay,

TABLE III
FEATURE VECTOR $\mathbf{x}_t$ ATTRIBUTES.

Category	Features
Behavioral	access time, frequency, session duration, request type, number of prior failed attempts, IP/device behavior pattern
Contextual analysis	forensic role, case phase, evidence sensitivity, role-phase divergence, hashed case-ID, token-access count, phase transition flag, version depth, provenance query count

TABLE IV
RISK STRATIFIED RESPONSE PROCEDURE.

Risk	Response
Low	Allow; follow standard provenance process flow; no Anomaly Ledger record.
Medium	Allow; alert superior; add anomaly record to queue.
High	Prompt additional authentication; alternatively put on hold; add anomaly record to queue.
Critical	Put on hold only for policy verification or repeat critical signs; add anomaly record to queue.

and the s_t is computed only after inference and never before for training purposes. The feature vector is formed using the pre-event profiles. The request at hand is only placed in the rolling state after all processes, i.e., feature extraction, inference, response selection, and queue placement, are completed and thus cannot impact its own score. Table III describes the various elements of the $\mathbf{x}_t$. Section VI-C evaluates the different candidate models for the f_θ .

2) *Risk level-dependent response*: In this case, we employ thresholds, that is, $\tau_1 < \tau_2 < \tau_3$ such that s_t is transformed into a risk level R_t that takes the following values:

$$R_t = \begin{cases} \text{low}, & s_t < \tau_1 \\ \text{medium}, & \tau_1 \leq s_t < \tau_2 \\ \text{high}, & \tau_2 \leq s_t < \tau_3 \\ \text{critical}, & s_t \geq \tau_3 \end{cases} \quad (7)$$

The described response is conservative in nature since neither one Medium nor High risk signal leads to permanent denial of access because a signal about a particular behavior does not necessarily mean its misuse, whereas multiple and Critical signals provide the basis for temporarily denying access according to the policy confirmation. Table IV summarizes the mapping.

3) *Asynchronous XAI and the Anomaly Ledger*: The flagged events are put into an *off-chain* queue with the anomaly ID, case number, pseudonymized user hash, role, phase, score, risk level, model hash, policy version, time stamp, and actual feature vector used during the inference process. The XAI agent will interpret this vector using TreeSHAP in an asynchronous fashion and create a report containing the ranked features, score, risk level, model hash, and a deterministic template explanation based on the report content. The report will be encrypted and saved *off-chain*, and its plaintext hash and random pointer will be recorded *on-chain*.

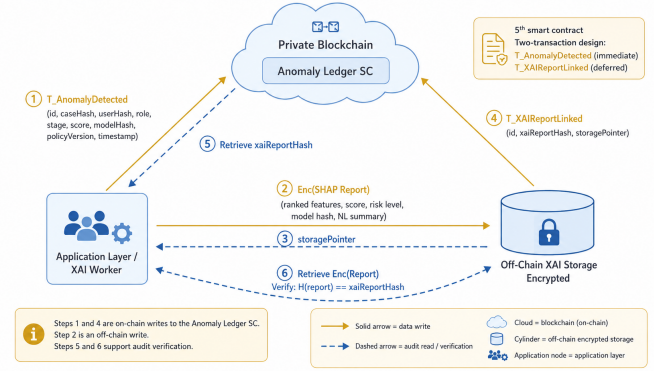


Fig. 2. Transaction-level data flow in the anomaly-aware extension. The medium/high or higher risk event is put in queue and access decision is sent (1); the worker then writes $T_{\text{AnomalyDetected}}$ (2), encrypts and saves the SHAP report off-chain (3), and writes $T_{\text{XAIRReportLinked}}$ (4) together with the report hash and pointer. Auditor reads on-chain record (5), decrypts the report and compares its hash with stored one (6).

Upon dequeuing a detected event, the worker verifies whether its record exists and, if needed, writes the detection transaction prior to computing SHAP:

$$T_{\text{AnomalyDetected}} = \langle id_t, caseHash_t, userHash_t, role_t, stage_t, s_t, modelHash_t, policyVersion_t, timestamp_t \rangle. \quad (8)$$

The same worker writes the linking transaction following the completion of the report generation, its hashing, encryption, and storage:

$$T_{\text{XAIRReportLinked}} = \langle id_t, xaiReportHash_t, storagePointer_t \rangle. \quad (9)$$

This ensures the recording of the detection record in case of any failure following the broadcast of the first transaction, although the non-durable nature of the in-memory queue does not guarantee the presence of the on-chain record at the time of the event dequeuing. The on-chain ID is pseudonymized due to privacy, $userHash_t = H(\text{salt} \parallel userID_t)$. To detect if any change occurred off-chain with the report after the storage, its integrity is ensured by calculating $H(\text{report}_t)$ and comparing it to the on-chain $xaiReportHash_t$, which will mismatch if there is any off-chain change. This is how the implemented transaction-level exchange looks like (Figure 2).

C. Summary: On-Chain, Off-Chain, Smart-Contract

Table V is a quick summary of where primary data and results of processing are stored in the original ForensiBlock system architecture and the modified one with anomaly awareness capabilities. Such segregation preserves the original storage architecture and ensures that the feature extraction and explanation generation do not get involved in the latency-critical blockchain execution pathway.

D. Descriptions of System Architectures

Figure 3 illustrates the entire procedure: the synchronous branch (in gray, with ForensiBlock unaltered, and white

TABLE V
DATA STORAGE AND ARCHITECTURAL DEPLOYMENT MATRIX.

Data / step	Location	Smart contract
Registration (public keys)	On-chain	Tokenized
Metadata about case, case stage and token list	On-chain	Case
Encrypted files with case evidence	Off-chain	—
Permission access details, RBAC-SA decisions	On-chain	Access Control
Provenance details (fully)	Off-chain	Provenance (verifies)
Merkle root (decentralized)	On-chain	Provenance
Feature vector $\mathbf{x}_t$, score s_t	Off-chain	—
Flagged case event (Medium/High/Critical)	Off-chain, in-memory	—
Anomaly detected event ($T_{\text{AnomalyDetected}}$)	On-chain	Anomaly Ledger
SHAP explanation report (complete)	Off-chain	—
Report hash and pointer ($T_{\text{XAIRReportLinked}}$)	On-chain	Anomaly Ledger

anomaly scoring) yields the result of the access decision, whereas the background worker executes the rest of the asynchronous explanation and ledger updates process illustrated in white on the right-hand side.

a) *Scope of the analyzed artifact*: The reference implementation is a *limited integration prototype*, which matters for the interpretation of all subsequent measurements. The evaluation environment includes all five contracts (Forensi-Block contracts, plus the Anomaly Ledger), and a contract lifecycle test suite covers the first four contracts on-chain. The actual experiments, however, lead the way through an execution-time implementation of the RBAC-SA policy matrix and the case lifecycle, the Anomaly Ledger being the only per-request called contract. The RBAC-SA baseline cost is thus the cost of an in-memory permission check, not an on-chain authorization operation, and the on-chain cost measured is thus the additional cost introduced by the extension, separated from the underlying provenance workflow. Preservation of the unchanged ForensiBlock provenance path is therefore guaranteed by construction rather than by an end-to-end experiment across all five contracts, which is left for future work.

V. IMPLEMENTATION

Every measurement in Section VIII comes from the artifact described here: a synchronous access pipeline, a queue-separated explanation worker, encrypted off-chain report storage, and a standalone Anomaly Ledger smart contract. This section records the versions, hosts and design decisions those numbers depend on.

A. Application System Stack and Environment

The application is built using Python 3.13.5, scikit-learn 1.9.0, XGBoost 3.3.0, SHAP 0.52.0, web3.py 7.16.0, and PyCryptodome 3.23.0. The Anomaly Ledger is coded in Solidity 0.8.20, with optimization enabled and executed for 200 runs. The private blockchain is run on Hardhat 2.28.6

and Node.js 22.22.0. The specific Python packages are fixed in the file called `requirements-lock.txt`; the minimum compatible versions are in `pyproject.toml`.

All the experiments have been conducted on one machine with an AMD Ryzen 7 5700U CPU (16 cores), 22 GiB memory, and Ubuntu 24.04.4. The Python program and database run on the same host machine. The Hardhat node is a separate local process with immediate mining. Consequently, the measurements of the blockchain properties are indicative of the local client and contract execution overheads, not the validator consensus overheads.

B. Characteristics and Rolling State Implementation

One canonical definition implies that the extractor, detector, dataset schema, and SHAP explainer have the same fifteen characteristics in the same order, eliminating possible discrepancies between training and serving time and between attributions and indexing. The entire set of characteristics is utilized in Layer B, whereas Layer A uses only the eight of them that can be computed on the basis of CERT, neglecting the remaining seven characteristics instead of zero-filling them and introducing the notion of evidence sensitivity as a file type surrogate and deviation of role-stage from user behavior baseline.

The preprocessed transform is a constructor argument of the access pipeline, and not something left behind by the training process. Train partition mean and standard deviation values are calculated only once for all vectors in canonical order and column names, and used for each vector before inferencing; the resulting vector after the standardization is the vector put in the queue, thus providing the explainer with the actual instance scored by the estimator. The pipeline built without the transform prints a message about the non-comparability of its scores with the offline evaluation.

There are three stores per user, case, and evidence identifier that implement behavioral state using Welford’s online algorithm by counting the number of observations, calculating mean and second central moment, along with auxiliary fields like last timestamp, device/IP, stage, sensitivity level, and version depth. The feature extractor observes the pre-event state and inference, risk classification, response selection, and enqueueing occur without the influence of the current request, which is added to the statistics after inference occurs. Reentrant locks protect the shared state and the reader sees only the snapshot of the state, not the live mutable state.

C. Queue and Explanation Worker

The events with medium, high, and critical severity are enqueued to a bounded severity partitioned queue with **one first-in, first-out (FIFO) deque per severity** (low deque is allocated but remains empty since the low severities are silently allowed). The queue capacity is 1000. Dequeueing is based on severity priority, and if the queue is full, it will drop the oldest event from the least severe class among the eligible severities, and reject the new event if all the other queued events have higher severities; saturation is therefore designed

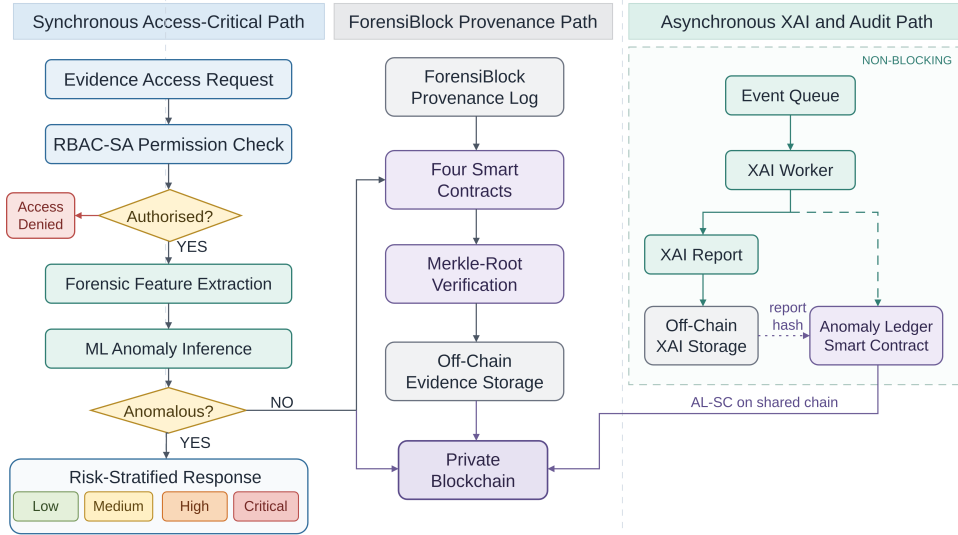


Fig. 3. Workflow implementation: same access decision and queue insertion, same ForensiBlock provenance processing, different XAI and audit processing in which the worker executes two Anomaly Ledger transactions. The gray parts are the same ForensiBlock; the white parts are the extension. In the experiment results, the gray path was running in process and Anomaly Ledger was executed on-chain.

to be lossy. The marked event holds the feature vector with which the inference was done, the anomaly score, the risk level, the hash of the model used, the version of the policy, the role, the stage, and the user hash, to enable SHAP to explain the decision made.

The worker provides direct processing, drain, and polling loop. The end-to-end testing performs the polling loop in a concrete background thread bound to the access pipeline queue and drains it during shutdown. The XAI overhead and queue throughput microbenchmarks are inherently synchronous by nature.

The worker makes an idempotency read before writing $T_{\text{AnomalyDetected}}$ in case there is no such record. It computes TreeSHAP on the feature vector, prepares a deterministic template summary of results, hashes the plaintext report with SHA-256, encrypts the report itself under AES-256-GCM, and writes the ciphertext to a filesystem-based report store under a new 32-byte random pointer. Finally, it will write $T_{\text{XAIReportLinked}}$ with the hash and pointer to the report. This method does not use any generative language model.

D. Security of Anomaly Ledger and Write Ordering

Anomaly Ledger is another contract and does not inherit or call any of the four existing contracts of ForensiBlock. The metadata of detection, hash of the report, pointer of storage and record/link status are saved in the mapping using anomaly identifier as key. Both external methods are only accessible to the owner or an approved writer. Guarded against duplicate anomaly identifier, linking to invalid records, and duplicate link. There are no re-entrant calls in the code as there are no external calls nor transfers of Ether done by the smart contract.

This ordering is weaker than a commit-at-detection design, in which the synchronous handler would write $T_{\text{AnomalyDetected}}$ before returning to the caller. Here the

worker writes it only after dequeuing, so an event can still be lost to process termination or queue overflow. What the ordering does guarantee is that once $T_{\text{AnomalyDetected}}$ has been committed, a later failure in SHAP computation, report storage, or the $T_{\text{XAIReportLinked}}$ write leaves an enumerable unlinked record rather than erasing the fact that the anomaly occurred.

E. Limitations of Implementation

Three limitations relate to the artifact itself rather than to the evaluation process. First, rolling profiles and queue data are held in process memory, so a restart returns them to a cold-start state, and there is no eviction strategy for per-entity state. Second, model artifacts are serialized with the Python pickle library and must therefore be treated as trusted inputs. Third, the range of contract inputs is validated mainly in the Python client, and the evaluated deployment runs the owner as the only authorized writer. See Section IX for a full discussion of prototype and evaluation limitations.

VI. EVALUATION

The evaluation answers two separate questions and therefore uses two corpora that are never pooled. Layer A asks whether the detector generalizes to authorized-user behavior it was not trained on, and is measured against an external benchmark. Layer B asks whether the integrated system — access decision, queue, explanation, ledger and audit — holds together end to end, and is measured on simulated ForensiBlock access logs. This section states the corpora, the split protocols, the candidate models and the measurement procedure behind each.

A. Synthetic and Benchmark Datasets

The datasets differ in feature structure, class balance and intent; therefore, they will be evaluated independently and

TABLE VI
SUMMARY AND CHARACTERISTICS OF EVALUATION DATASETS.

	Layer A	Layer B
Source	CERT r4.2	Generated
Events	330,452	50,000
Unit	user-day	access request
Features	8 of 15	15
Users	1,000	20
Insider users	70	—
Cases	—	10
Positive events	966	2,000
Positive rate	0.29%	4.00%

never pooled together. **Layer A evaluates the detection accuracy of the eight-feature adaptation of CERT r4.2 [14].** Layer B employs all 15 features and exercises the entire access, explanations, queuing, blockchain, scaling, and auditing process.

1) *Layer A: CERT r4.2*: There are 330,452 user-day instances of 1,000 users in Layer A, including 966 positive instances generated by 70 insiders. CERT is an artificial dataset for organizational activity, not forensic provenance data. Therefore, CERT is used as an authorized user-behavior benchmark, not as a proof of evidence access dataset.

Eight out of 15 features, `access_hour`, `access_frequency`, `session_length`, `request_type`, `device_or_address_pattern`, `role`, `evidence_sensitivity`, and `role_stage_deviation`, are available to us. These seven features — `prior_failed_attempts`, `case_stage`, `case_identifier`, `token_access_count`, `stage_transition`, `evidence_version_depth` and `provenance_query_count`, are not present in CERT and therefore are not default, zero-filled, or synthesized. Two out of the eight features that we do have require qualifications; `evidence_sensitivity` is defined through a proxy, file-extension, and `role_stage_deviation` is re-defined to mean deviation from the user’s rolling behavior because there is no investigation stage feature in CERT. Consequently, Layer A evaluates an eight-dimensional CERT adaptation instead of the full ForensiBlock feature set.

The positive rate is around 0.29%, which makes AUPRC, precision, recall, false positive rate, and operational alert load more relevant than accuracy. Layer A is used solely for detection performance evaluation because the blockchain, queue, latency, XAI storage, and audit layers require full-schema access paths.

2) *Layer B: ForensiBlock-style access logs simulation*: Layer B is produced through the reproduction of the lifecycle of the ForensiBlock case, but rather than writing feature vectors in their entirety, Layer B is generated. The world comprises 20 users and 10 cases; each case includes three items of evidence of varying levels of sensitivity along with a derived evidence chain of two levels. Each request creation is subject to the same feature extraction and rolling state algorithms as the live synchronous path such that access frequency, role-stage mismatch, token access count, and provenance query

TABLE VII
INJECTED LAYER-B ANOMALY CASES.

Scenario	Major impacted features
Off-hours access	<code>access_hour</code>
Excessive downloads	<code>access_freq</code> , <code>token_access_count</code>
Wrong-stage access	<code>role_stage_deviation</code> , <code>stage_transition_flag</code>
Credential misuse	<code>prior_failed_attempts</code> , <code>device_ip_pattern</code>
Abnormal provenance rate	<code>provenance_query_count</code> , <code>evidence_version_depth</code>

count accumulate through user, case, and evidence history using a synthetic clock tick between work items using a uniformly sampled period of 30 to 600 seconds.

Normal requests involve RBAC-SA-permitted role-stage pairs. Anomalous requests involve RBAC-SA-permitted role-stage pairs because the dataset concerns authorised anomalous activity and not permission breaches. Five scenarios are inserted as summarized in Table VII.

The excess download and anomalous provenance scenarios are created using short bursts, wherein each row uses one user, selected anew for each burst. On the other hand, single event injections include off-hours access and credential abuse, using a new actor for each event. Wrong stage access is also a single event injection, which is a deterministically selected actor for each row of the scenario by the generator, making each row of this scenario be associated with one user; this is an error in the generator which is addressed through the Layer-B leakage analysis in Section VIII-B2. The default data set has 50,000 events, of which only 2,000 are positive, making 4% of all the events. Both normal and anomalous events get shuffled prior to clock advancement in the synthetic setting and the generation process can be reproduced with the configured seed. A row comprises the canonical 15 features, the ground truth label, and the metadata of the event, user, case, stage, timestamp, and scenario.

Layer-B checks that the integration of the entire 15-feature pipeline, the asynchronous explanation path, and the on-chain audit path is correct. The excellent results achieved by Layer-B do not serve as the evidence of the generalization capability towards unseen insider behaviors since anomalies are artificially introduced through the feature mappings.

3) *Version sensitive to feature noise*: There is an optional version that adds independent zero-mean Gaussian noise with $\sigma = 0.15$ to the fifteen numerical features, leaving the metadata and labels intact; this is a deterministic transformation that is conditional on the random seed and is off by default. This test is considered a feature noise sensitivity test and is not inherently a harder corpus as the addition of noise does not necessarily make class separation any more difficult. All 50,000 records are left untouched, as are the 2,000 positive labels, meaning any change in performance is due solely to feature noise.

B. Split Protocol and Preprocessing

Since both datasets involve repeating entries for users with known bounds, a random split would result in some users being present in the train and test set. The partitions are made in 70/15/15 ratio as per one of the following two user-aware methods:

- *First seen ordered deterministic user-disjoint split.* Users are sorted based on their first seen time and then split disjointedly. This is not necessarily a temporally ordered split between events and does not mean that all test events occur after all the training events.
- *Seeded user-disjoint group shuffling.* Users are split based on a seeded permutation.

The strict temporal protocol alone is used but not the combination of it with user-disjointness; the repetition of the statistical test is performed through the group shuffle protocol.

a) *User disjoint splitting:* In user disjoint splitting, all rows from a burst are placed into a partition because each burst type anomaly involves a single user. The categorical variables are either one hot or ordinal encoded and numerical features are standardized with the help of statistics of the training partition only.

b) *Train-serve consistency:* Same fitted transformation is used for offline as well as online scoring. The mean and standard deviation of the training split produced by the preprocessing pipeline are used in the runtime access path that standardizes the extracted feature vectors before making predictions. As a result, the model selection process, thresholds calculated based on the validation and test set performance, risk classification during runtime, and TreeSHAP importance calculations use the same feature space. The standardized feature vector used for the model prediction is the one extracted in the flagged event and presented to the explainer without any changes. All runtime-dependent results in Section VIII were produced under this configuration.

The problem of class imbalance is handled at the model level, not through resampling. **The synthetic minority over-sampling technique (SMOTE) cannot be applied, because fractional synthetic values would not make sense for variables that are either discrete or ordinal.** Random Forest uses balanced class weights while XGBoost uses `scale_pos_weight =  $n_{\text{neg}}/n_{\text{pos}}$` , where both are calculated only on the training labels. Isolation Forest optimizes contamination on validation data. Rule-based baseline uses equal weighting.

C. Candidate Models and Model Comparison

Four models are compared under the same split. The *rule-based* baseline follows pre-defined thresholds where access hour does not lie between [7, 21], access frequency is greater than 20, or there are more than three previous failures, and calculates the percentage of relevant triggered rules. Each of the rules can only apply if the respective feature is there, which means that Layer A checks for the first two features and Layer B checks for all three features, previous unsuccessful attempts being one of the seven features not available with

CERT. The denominator changes to make it one detector for two feature availabilities, rather than two differently tuned detectors. For *Isolation Forest* [17], number of estimators and contamination are selected through exhaustive grid search using validation set with F_1 -score, while no cross-validation is used. *XGBoost* [19] finds the number of estimators, depth, and learning rate through exhaustive search based on the validation dataset, with the rate of subsampling set to 1.0, and also supports TreeSHAP [24]. *Random Forest* [18] selects the number of estimators, the maximum depth, and the minimum number of samples per leaf through the same exhaustive validation-set search. The committed grids for Random Forest and XGBoost each contain eight configurations.

Eligibility begins with the gate where the false positive rate is no more than 0.05 and mean inference latency no more than 10 ms; models that satisfy the eligibility criteria are sorted by the validation AUPRC and, for models with AUPRC within 0.01 of each other, by recall, then latency, then model name. F_1 score is computed but not used for the final selection.

D. Statistical Protocol and Metrics

The performance of the detection system is measured in the following ways: precision, recall, F_1 , false positive rate, AUROC, and AUPRC. AUROC and AUPRC are computed based on the continuous score. The remaining four are computed based on the label prediction made by the native model — the predictions of the supervised classifiers, decision of the Isolation Forest using contamination rate, and the combination of rule-based trigger.

Metrics are provided on $N = 30$ seeded user disjoint splits. Percentile bootstrap intervals at the 95 percentiles level are computed based on 10^4 repetitions of per-split values, and pair-wise comparisons of the models using Wilcoxon signed rank test on split-level vector are done using a Holm-Bonferroni correction on all six model pairs and rank biserial effect size measure. Paired Wilcoxon tests are used to analyze latency differences between RBAC-SA-only and the entire synchronous path.

E. Risk Threshold Calibration and Operational Load

The risk level thresholds of (7), $\tau_1 < \tau_2 < \tau_3$, are defined by the quantiles of the validation negative score distribution for false positive rate of 5%, 1%, and 0.1% for Medium, High, and Critical. Statistics for tiers are computed on the test set only.

This is only applicable if validation, testing, and runtime scores are produced within the same feature space transformation, which is guaranteed by the common fit transformation in Section VI-B. The harness outputs the number of events, number of true positives, precision per tier, percentage of all positives, total alerts, recall and alerts per 1,000 accesses per tier. Low-risk requests are automatically passed without making an anomaly queue entry nor generating any Anomaly Ledger entries. Medium, High and Critical risks are the only risk levels to be processed asynchronously.

Threshold robustness is assessed by scaling the three false positive bounds by values of 0.2, 0.5, 1.0, 2.0 and 5.0. A sensitivity analysis (not presented here) applies deviations of $\pm 10\%$ to each threshold while keeping the others constant and ignoring any order-breaking deviation. The results from these tiers are simulations of deployment on Layer A, because no active response policy is in place for CERT. Layer B applies the response policies from Table IV.

F. Latency, Queue and End-to-End Measurements

Three paired comparisons establish what the extension costs at run time. The first is done between RBAC-SA alone and the full synchronous anomaly-aware path; the second is done between the four detectors under identical partitions; and finally, the last comparison is that of the detection and linking transactions of the Anomaly Ledger contract alone.

The boundaries of the synchronous timing microbenchmark include RBAC-SA, pre-event feature extraction, inference, risk classification, response choice, medium or higher queue insertion, response hooks, and the next rolling state update, while SHAP and blockchain writes are excluded from the boundary. Latency is measured over $n = 200$ request pairs and includes mean, median, P95, and P99 latency measurements for RBAC-SA latency separately. Overhead is measured over $n = 20$ flagged events and includes TreeSHAP, reporting, encryption and storage, and total overheads. Both are synchronous microbenchmarks, but there is an asynchronous end-to-end dimension that measures the number of enqueued, dropped, finished, detected, and linked events.

Each severity has a deque-based queue with capacity 1,000, which prioritizes Critical requests before draining High and Medium; drops count is recorded, and the queue is non-durable. Scalability was tested at 10, 50 and 100 req/sec for one second, so the provided P99 values are characteristic of tail performance in short bursts, not steady-state tail latency. Standardization includes an $O(d)$ element-wise transformation, which does not dominate latency measurement. All the latencies, scalability and end-to-end metrics in Section VIII have been measured with standardization, so the level of risk, queue operations, response hooks and load on the worker are determined in the feature space after standardization.

G. Assessment of Smart Contract and Blockchain

The Anomaly Ledger is run in a single-node Hardhat blockchain with automated mining. Receipts from mined blocks are used to measure gas usage and calldata based on the ABI-encoded transaction input data. Detection and linking of transactions are logged separately. The amount of calldata used is an estimation of transaction-input growth rather than contract state or log event size growth. Throughput is estimated in 30-second intervals for 10, 50 and 100 transactions/second using separate counters for the number of transactions submitted and confirmed, with an estimation of the completion interval, $\Delta t = t(T_{\text{XAIRreportLinked}}) - t(T_{\text{AnomalyDetected}})$, i.e. time between the detected and linked transactions, for $n = 50$. In addition to this, there is no consensus round done

in the local automining network, and hence this metric will measure local **Ethereum Virtual Machine (EVM)** and local client execution and not a multi-validator permissioned setup.

H. Audit Trail Verification and XAI

The XAI evaluation is concerned with the time taken for computing the TreeSHAP values, report generation, encryption/storage of data. The worker describes the feature vector that is stored while making inference, maintains the correct order of features, computes the SHA256 hash of the plaintext report, then encrypts it using AES-GCM algorithm. The retained vector is the standardized vector which was scored, such that the attributions are measured in the space in which the estimator was trained; hence, the feature values reported in the output documents are measured in standard deviation units with respect to the training split and not in raw units. There is no corpus-wide top- k consistency measure for the Layer-B attribution results in the five scenarios presented; hence, no attribution consistency results are reported here.

The audit verifier lists anomaly reports from a recorded start block, rehashes the model artifact hash from disk, retrieves and decrypts the corresponding reports, verifies the plaintext report hash, verifies the policy version and lists unlinked reports; automated tests cover modifications on ciphertext, fake model hashes, wrong policy versions and interruption of workers. The lack of a retry/dead-letter process and memory queue cause an event to be lost before the detection of the anomaly transaction. As a result, “broadcast” is insufficient to prove durability since the record will be ensured available only after the transaction confirmation on chain. Hence any failure between the moment of submission and on-chain confirmation should be dealt with via receipt validation and idempotent retry.

The event holds a model hash; however, the worker does not validate the hash with the currently loaded model to compute SHAP values. Hence, the prototype lacks cryptographic proof that the event will be explained by the very same model instance which has computed its score initially.

VII. ALGORITHM

The suggested workflow separates the access-sensitive decision-making process from the generation of the explanations and the closing of the ledger. Algorithm 1 runs RBAC-SA verification, feature extraction, anomaly scoring, risk categorization, action selection, event passing to the worker, and rolling state updating. Algorithm 2 processes only events with Medium, High, and Critical risks and completes the two-transaction auditing workflow.

Let r_t represent an access request at the time of detection t_t by user u_t of case c_t with evidence item e_t , role $role_t$, and investigation phase g_t . Let $m_t = H(f_\theta)$ be the hash of the specific model artifact used for scoring. The passed event is

$$evt_t = \langle id_t, c_t, h_u, role_t, g_t, s_t, R_t, m_t, policyVer, t_t, \mathbf{x}_t \rangle. \quad (10)$$

Here, $h_u = H(salt \parallel u_t)$ is the pseudonymous user id, while R_t and $\mathbf{x}_t$ are the risk level and feature vector, respectively,

Algorithm 1 Anomaly-Aware Access Control: Synchronous Path

Require: Access request r_t ; model f_θ ; fitted train-partition statistics (μ, σ) ; thresholds $\tau_1 < \tau_2 < \tau_3$; permission table $\mathcal{P}$; statistics store $\mathcal{S}$; bounded queue $\mathcal{Q}$; policy gate ρ

Ensure: d_t is an access decision

Phase 1: Permission check using RBAC-SA $O(1)$

- 1: **if** RBACSA($\mathcal{P}, r_t$) = DENY **then**
- 2: **return** DENY $\triangleright$ no feature or model cost
- 3: **end if**

Phase 2: Feature extraction from pre-event state $O(d)$

- 4: $\mathbf{x}_t \leftarrow \text{EXTRACTFEATURES}(r_t, \mathcal{S})$ $\triangleright d = 15$; no current-event update yet

Phase 3: Scoring anomalies $O(TD)$

- 5: $\mathbf{x}_t \leftarrow (\mathbf{x}_t - \mu) \oslash \sigma$ $\triangleright$ train-fitted scaling, elementwise
- 6: $s_t \leftarrow f_\theta(\mathbf{x}_t)$ $\triangleright s_t \in [0, 1]$

Phase 4: Risk and response decision $O(1)$

- 7: $R_t \leftarrow \text{CLASSIFYRISK}(s_t, \tau_1, \tau_2, \tau_3)$
- 8: $p_t \leftarrow \text{POLICYCONFIRMED}(\mathcal{S}, c_t)$
- 9: $q_t \leftarrow \text{REPEATEDSEVERE}(\mathcal{S}, u_t)$
- 10: $d_t \leftarrow \text{DETERMINERESPONSE}(R_t, p_t, q_t, \rho)$

Phase 5: Event construction and non-blocking enqueue $O(1)$

- 11: **if** $R_t \neq \text{Low}$ **then**
- 12: $\text{id}_t \leftarrow \text{UUID}; h_u \leftarrow H(\text{salt} \parallel u_t)$
- 13: $\text{evt}_t \leftarrow \text{BUILDEVENT}(r_t, s_t, R_t, m_t, \mathbf{x}_t)$
- 14: $\eta_t \leftarrow \mathcal{Q}.\text{ENQUEUE}(\text{evt}_t)$ $\triangleright \eta_t$ does not alter d_t
- 15: **end if**

Phase 6: Response hooks $O(1)$

- 16: $\text{INVOKEHOOKS}(d_t, u_t, c_t, s_t)$ $\triangleright$ supervisor alert, step-up or block

Phase 7: Update in rolling state $O(d)$

- 17: $\text{RECORDRISKEVENT}(\mathcal{S}, u_t, R_t)$
- 18: $\text{UPDATESTATISTICS}(\mathcal{S}, r_t)$ $\triangleright$ executes for each RBAC-SA-authorized request
- 19: **return** d_t

used in the inference process. The risk level and feature vector do not form part of the information in the Anomaly Ledger; rather, they exist only in the process and are not committed to the Anomaly Ledger. The worker proceeds to compute the report hash h_r and storage pointer ptr_t of the second transaction. The timestamp t_t is that of the detection path, not of the enqueue/dequeue/write-to-ledger path.

Response mapping by the policy engine, given in (11), is Allow when $R_t = \text{Low}$, AllowSupervisor when $R_t = \text{Medium}$, StepUp when High, and TempBlock when Critical and there is confirmation or a repeated-severe case with the policy gate set; and StepUp for all other Critical events.

$$d_t = \begin{cases} \text{ALLOW}, & R_t = \text{Low}, \\ \text{ALLOWSUPERVISOR}, & R_t = \text{Medium}, \\ \text{STEPUP}, & R_t = \text{High}, \\ \text{TEMPBLOCK}, & R_t = \text{Critical}, \rho \wedge (p_t \vee q_t), \\ \text{STEPUP}, & R_t = \text{Critical}, \neg[\rho \wedge (p_t \vee q_t)], \end{cases} \quad (11)$$

In this case, p_t refers to policy confirmation, q_t refers to a repeated-severe condition, and ρ refers to the configuration gate used for temporary blocking in the event of confirmation or a repeated Critical situation.

Algorithm 1 computes $\mathbf{x}_t$ using the pre-event statistics. Updating of the rolling state is done after scoring, enqueueing, and response-hook dispatching. If the enqueue operation fails, the status of the handoff reported changes, but the response

Algorithm 2 XAI and Anomaly-Ledger Worker (Asynchronous)

Require: Queue $\mathcal{Q}$; model snapshot f_θ ; ledger interface $\mathcal{B}$; encrypted report store $\mathcal{O}$

Ensure: Anomaly record and corresponding off-chain explanation for each event processed

- 1: **while** $\mathcal{Q}$ is not empty **do** $O(1)$
- 2: **Step 1: Dequeue based on severity** $O(1)$ client work
- 3: $\text{evt}_t \leftarrow \mathcal{Q}.\text{DEQUEUE}$
- 4: **Step 2: Idempotency read** $O(1)$ client work
- 5: $\text{rec} \leftarrow \mathcal{B}.\text{READRECORD}(\text{id}_t)$
- 6: **if** $\text{rec}.\text{linked}$ **then**
- 7: **continue** $\triangleright$ idempotent no-op
- 8: **end if**
- 9: **Step 3: First anomaly-record transaction**
- 10: **if** $\neg \text{rec}.\text{exists}$ **then**
- 11: $T_{\text{AD}} \leftarrow \langle \text{id}_t, c_t, h_u, \text{role}_t, g_t, s_t, m_t, \text{policyVer}, t_t \rangle$
- 12: $\text{SUBMITANDCONFIRM}(\mathcal{B}, T_{\text{AD}})$
- 13: **end if**
- 14: **Step 4: Calculation and ranking of TreeSHAP**
- 15: **require** $\mathbf{x}_t \neq \emptyset$
- 16: $\phi_t^\downarrow \leftarrow \text{RANKEDTREESHAP}(f_\theta, \mathbf{x}_t)$ $\triangleright O(TLD^2 + d \log d)$
- 17: **Step 5: Construction of deterministic report** $O(d)$
- 18: $\text{summary}_t \leftarrow \text{RENDERSUMMARY}(\phi_t^\downarrow, s_t, R_t, \text{role}_t, g_t)$
- 19: $\text{report}_t \leftarrow \text{BUILEREPORT}(\phi_t^\downarrow, s_t, R_t, m_t, \text{summary}_t)$
- 20: **Step 6: Hash, encrypt and store** $O(|\text{report}|)$
- 21: $\text{ptr}_t \leftarrow \text{SERIALIZECANONICAL}(\text{report}_t)$
- 22: $h_r \leftarrow H(\text{ptr}_t)$ $\triangleright$ plaintext hash
- 23: $\text{ctr}_t \leftarrow \text{AESGCMENCRYPT}(\text{ptr}_t)$
- 24: $\text{ptr}_t \leftarrow \mathcal{O}.\text{STORE}(\text{ctr}_t)$
- 25: **Step 7: Explanation link transaction**
- 26: $T_{\text{XL}} \leftarrow \langle \text{id}_t, h_r, \text{ptr}_t \rangle$
- 27: $\text{SUBMITANDCONFIRM}(\mathcal{B}, T_{\text{XL}})$
- 28: **end while**

choice remains unchanged, as well as the update step, which is not hidden anymore.

The train partitioning standardization is made in the scaling line (Line 5), as before scoring, in order to make sure that the vector $\mathbf{x}_t$ is in the feature space where it was fitted and is the same vector that has been put on the queue.

In the blockchain client, the logical fields of T_{AD} are encoded: the string ID is encoded to `bytes32`, and the anomaly score is the integer `round(106 $s_t$ )`. For each `SUBMITANDCONFIRM` action, a transaction is submitted, which does not proceed further until the transaction receipt gets mined. T_{XL} contains the anomaly ID, the plaintext hash of the report, and the fresh 32-byte storage pointer for the report. The encrypted report is stored off-chain.

Finite durability thus arises from the order of actions in Algorithm 2. Following the successful submission of T_{AD} , any future failures of SHAP, storage, or link write are now an enumerable unlinked anomaly record. $\mathcal{Q}$ is a non-durable data structure, hence between enqueue/dequeue and ledger write, any failure will destroy the evidence. Another point of failure that could cause an orphaned report that will not be automatically cleaned up is failure between the encrypted storage and T_{XL} . Ledger state and contract replay guards make retries idempotent once the initial transaction has been logged.

The queue is a bounded queue partitioned on severity and is composed of four FIFO dequeues specific to each severity level. Dequeue always picks the most severe entry in the queue

and preserves FIFO ordering for entries of the same severity. If the queue is full and the least severe entry in the queue is no more severe than the incoming event, then that entry is removed; otherwise, the incoming event is dropped. The four static buckets ensure that enqueue and dequeue are $O(1)$. Enqueue and Dequeue in the queue are serialized through a re-entrant lock.

There are two ways of calling the workers during the evaluation process. The XAI-overhead and queue-drain micro-benchmarks execute the worker synchronously through the drain function. However, the end-to-end experiment executes Algorithm 2 asynchronously in the background. This is reported separately since it is only in this scenario that asynchronous handoff occurs based on the access decision.

A. Complexity Analysis

Let d be the dimension of the feature vector, T the number of trees, D the max depth of a tree and L the max leaves of a tree. Define $|\mathcal{P}|$ as the number of permission entries; $|\mathcal{U}|$, $|\mathcal{C}|$, and $|\mathcal{E}|$ as the number of users, cases, and evidences respectively that are being monitored; $|\mathcal{Q}|$ as the queue size; $|\mathcal{M}|$ as the in-memory model size; $|H|$ as the total history size in raw form; W as the number of observations kept by a windowed alternative approach; $|report|$ as the serialized report size; and n as the number of flagged events kept for some time.

Synchronous approach. The RBAC-SA check is an expected $O(1)$ access of permission table. Instead of browsing the combined user, case, and evidence histories, the feature extraction relies on d maintained statistics. History-dependent feature extraction becomes $O(d)$ instead of $O(|H_u| + |H_c| + |H_e|)$. Rolling update becomes $O(d)$ with online Welford statistics [20]. Inference of the tree ensemble considers at most T trees with depth D , so its complexity is $O(TD)$. Mapping the risk, dispatching the hook, and operations on the fixed-bucket queue take $O(1)$ each. Thus, synchronous computation time is

$$\mathcal{T}_{\text{sync}} = O(d + TD). \quad (12)$$

Asynchronous computation path. Exact TreeSHAP algorithm has complexity of $O(TLD^2)$ per event [24]. Ranking of d contribution items takes $O(d \log d)$. Rendering the deterministic report has complexity $O(d)$. Serialization, AES-GCM encryption, and writing reports are linear in their size. The two blockchain queries need continuous client-side encodings and signatures; the latency of external confirmation is empirical, not asymptotically as a CPU term. Therefore, the asynchronous time complexity is

$$\begin{aligned} \mathcal{T}_{\text{async}} &= O(TLD^2 + d \log d + |report|) \\ &= O(TLD^2). \end{aligned} \quad (13)$$

which simplifies to $O(TLD^2)$ if the value of d is fixed.

Total storage and memory. Persistent online memory is

$$S_{\text{persistent}} = O(|\mathcal{P}| + d(|\mathcal{U}| + |\mathcal{C}| + |\mathcal{E}|) + |\mathcal{Q}| + |\mathcal{M}|), \quad (14)$$

while the synchronous auxiliary memory required is $O(d+D)$, and for each event of a worker, it is $O(D^2 + d + |report|)$. Report storage is an off-chain resource usage with complexity $O(n|report|)$, and Anomaly-Ledger state size and log length is $O(n)$.

This complexity stems from the following five design decisions: the use of rolling statistics saves rescans and reduces entity state from $O(W)$ for the window-based implementation to $O(d)$; RBAC-SA denial stops processing the request prior to feature generation and inference; tree ensembles allow bounded inference using tables without a GPU; TreeSHAP and ledger finalization are done off the critical path of access; and the severity queue maintains a fixed memory bound and favors storing high-severity events.

VIII. RESULTS

The two layers are reported separately because their goals are different (Section VI-A). Hence, the higher values of Layer B correspond only to performance on the injected tasks and do not reflect any generalized ability to perform in the real world.

All detection results represent averages across 30 re-splits using the same group-shuffle method with 95% confidence intervals based on 10^4 percentile bootstrap resamples, unless otherwise specified. Paired comparisons use Wilcoxon signed-rank test with Holm–Bonferroni correction across all six pairs of models per metric. This is due to the fact that these artifacts come out of the same group-shuffle procedure regardless of the command-line split procedure and hence are reported only once; protocol-specific comparisons are identified separately and never pooled.

A. Layer A: CERT r4.2 Detection

The dataset Layer A consists of 330,452 user–days with 1,000 users and 966 positive instances with a positive rate of about 0.29%. Only the eight behavioral features supported by CERT are taken into account. Since the ForensiBlock provenance features are not fabricated, Layer A only evaluates the detection performance.

1) *Models comparison:* Table VIII and Fig. 4 present the six main metrics. Accuracy metric is omitted, as it is skewed by the majority class. Rule-based classifier detects almost all instances, however, at an unacceptably high false-positive rate, sitting at the no-skill ranking floor (Table VIII). The Isolation Forest manages to lower the False Positive Rate to 0.021 ± 0.005 and achieve AUROC of 0.919 ± 0.008 , but is still limited by having an AUPRC of only 0.033 ± 0.005 — which illustrates the weakness of AUROC on its own on this particular dataset: one can have a high ranking of negative instances while still having poor precision at the analyst’s cut-off point.

None of the supervised ensemble methods is clearly dominant. Random Forest gives the best precision (0.109 ± 0.028), F_1 (0.159 ± 0.036) and the least False Positive Rate (0.011 ± 0.002); on the other hand, XGBoost gives the best Recall (0.588 ± 0.065), AUROC (0.953 ± 0.010) and AUPRC

(0.257 ± 0.047), while the former represents an estimated 80 $\times$ lift from the no-skill baseline at a test positive rate of 0.32%. Thus, Layer A is the trade-off between precision and recall, where Random Forest is conservative compared to XGBoost, which finds more insiders and ranks better.

The supplemental material further includes the precision–recall plots of the aforementioned experiments. Here the precision is rather low since the positive class is rare; however, the curve of the XGBoost method shows the best rankings within most of the recall range, whereas the rule-based curve stays near the prevalence baseline.

2) *Statistical comparison between pairs*: Figure 5 reports the effect size for the matched-pairs rank-biserial correlation of each model pair, with asterisks denoting significance at $\alpha = 0.05$ following Holm–Bonferroni correction. The paired test demonstrates that the Random Forest versus XGBoost comparison is consistent across models and not a result of one split only. The AUPRC of XGBoost is significantly higher compared to Random Forest (Holm-corrected $p = 8.33 \times 10^{-7}$; the effect size is -0.910 when Random Forest is the row model). XGBoost also has significantly higher recall (Holm-corrected $p = 1.03 \times 10^{-5}$; effect size -1.000).

But still, Random Forest demonstrates significantly greater precision ($p_{\text{Holm}} = 1.98 \times 10^{-4}$), higher F_1 score ($p_{\text{Holm}} = 3.74 \times 10^{-3}$, effect $+0.591$), and lower false positive rate ($p_{\text{Holm}} = 1.68 \times 10^{-8}$). Difference in the AUROC values between the two ensembles does not have statistical significance ($p_{\text{Holm}} = 0.129$). Practically speaking, this is a compromise between the two models. In case that the goal is to detect and correctly rank as many suspicious cases as possible, then XGBoost should be chosen (high recall, AUROC, and AUPRC values); otherwise, Random Forest would suit better (high precision, F_1 , low FPR). It all comes down to preference, which is dependent on whether the operator is more worried about missing insider behavior than about alert fatigue.

3) *Identity-leakage control*: Another table presents the validation splits for the user-disjoint and strict-temporal run validation partitions of both layers; it is a leakage control and not a substitute for the 30-re-split test results. With strict temporal partitioning, the users are repeatedly present in the training set, validation set, and test set. The AUPRC of Layer A for XGBoost rises from 0.099 to 0.335 and for Random Forest from 0.069 to 0.241 (about 3.4 times and 3.5 times, respectively). Isolation Forest decreases instead, from 0.073 to 0.032. As such, it is possible that the supervised methods could exploit user-specific patterns that are not present in the label-free algorithm, and the user-disjoint methodology remains the main protocol for Layer A.

B. Layer B: Detection within Simulated ForensiBlock Logs

Layer B contains 50,000 access requests, 2,000 positive instances, 20 users, 10 cases, and all 15 features. The anomaly injection processes have been performed using a predefined mapping of features; hence, Layer B is used primarily for scenario evaluation purposes, and the detection scores are

calculated using performance in the predetermined scenarios and not using real-world challenges.

1) *Models comparison*: According to Table IX and Fig. 6, the performance of both ensembles is much better compared to that of the two baseline methods. For the Random Forest, we observe precision 0.996 ± 0.003 , $F_1 = 0.778 \pm 0.066$, AUROC 0.924 ± 0.040 , AUPRC 0.745 ± 0.060 , and an FPR rounding to zero; as for the XGBoost model, it produces almost the same recall (0.671 ± 0.068 versus 0.669 ± 0.068) and F_1 score (0.768 ± 0.069), however, its AUPRC is lower (0.712 ± 0.059). In the paired comparison, the Random Forest demonstrated a significantly higher AUPRC ($p_{\text{Holm}} = 1.12 \times 10^{-8}$, effect $+0.996$) and AUROC ($p_{\text{Holm}} = 3.48 \times 10^{-3}$, effect $+0.596$), with no significant difference in precision, recall, F_1 , and FPR. Thus, Random Forest results in a more accurate ranking of events. The two methods have the same statistical power at the native operating point of each one, corresponding to the same trade-off between precision and recall seen in Layer A: Random Forest sacrifices some detection rate for close-to-no false alarms, whereas XGBoost sacrifices a bit of precision for improved ranking of hard cases.

Wide intervals of Layer B are not an accidental outcome. The generator defect binds every `wrong_stage_access` event to one and the same user, so user-disjoint splitting leaves the scenario either entirely absent from a test fold or present in it as a single block, which makes the recall distribution bimodal. The defect is preserved as a disclosed limitation of the corpus. Results include description of the generated corpus as evaluated.

The model selection performed during protocol-specific runs also falls under a noisy 10 ms latency gate; that is, Random Forest has been rejected in favor of XGBoost in the user-disjoint run, where its latency was measured to be 16.49 ms, but selected on AUPRC in the strict-temporal run, where its latency was measured to be 9.01 ms. This implies that there is sensitivity to latency measurement around the gate rather than any general effect caused by the split protocol.

2) *Leakage control using protocol*: The same supplementary table includes validation for Layer B; the strict-temporal AUPRC is higher by about 2.3%, 4.7%, and 19.4% for Random Forest, XGBoost, and Isolation Forest, respectively. Unlike Layer A, the label-free model improves the most, and the generator defect causes the two protocols to evaluate different scenario sets. These observations show protocol sensitivity with a small 20-user generated corpus, not identity leakage; Layer A offers the cleaner leakage control.

3) *Detectability by scenario*: The most insightful finding from Layer B is related to the break-down of the scenario results in Table X, which were computed from the strict-temporal test partition using the Random Forest technique and contain all five scenarios. For misuse of credentials, wrong-stage access, abnormal provenance rate and too many downloads all the test events are classified as Critical. In contrast to that, off-hours access consists of no Critical events and 20.6% of Medium or higher class events among 63 total test events.

TABLE VIII
CERT R4.2 LAYER A DETECTION. VALUES ARE MEAN $\pm$ 95% CONFIDENCE INTERVAL REPORTED OVER 30 GROUP-SHUFFLE RE-SPLITS. THE BEST MACHINE LEARNING RESULT IN EACH COLUMN IS HIGHLIGHTED IN BOLD.

Model	Precision	Recall	F_1	AUROC	AUPRC	FPR
Rule-based	0.003 $\pm$ 0.001	0.843 $\pm$ 0.057	0.005 $\pm$ 0.001	0.539 $\pm$ 0.037	0.004 $\pm$ 0.001	0.830 $\pm$ 0.012
Isolation Forest	0.040 $\pm$ 0.010	0.290 $\pm$ 0.062	0.063 $\pm$ 0.013	0.919 $\pm$ 0.008	0.033 $\pm$ 0.005	0.021 $\pm$ 0.005
Random Forest	0.109 $\pm$ 0.028	0.419 $\pm$ 0.067	0.159 $\pm$ 0.036	0.937 $\pm$ 0.020	0.174 $\pm$ 0.043	0.011 $\pm$ 0.002
XGBoost	0.075 $\pm$ 0.017	0.588 $\pm$ 0.065	0.127 $\pm$ 0.026	0.953 $\pm$ 0.010	0.257 $\pm$ 0.047	0.022 $\pm$ 0.003

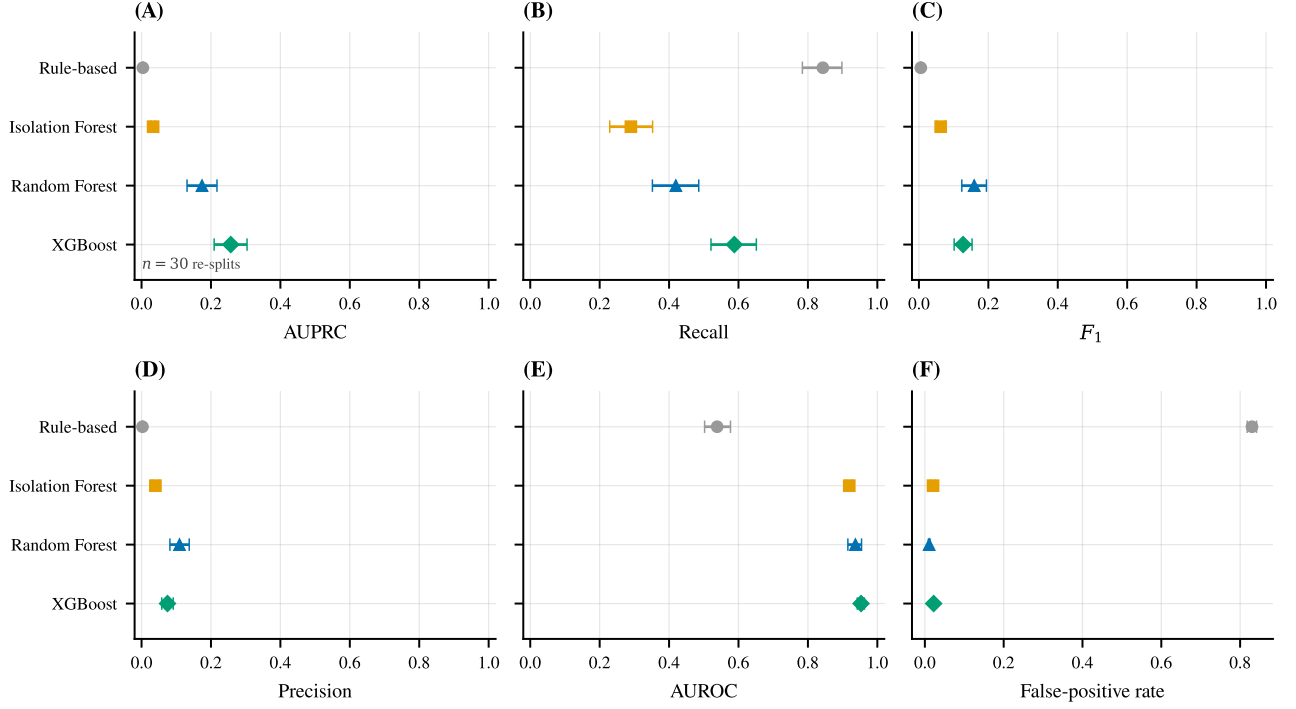


Fig. 4. Mean detection metrics and 95% confidence intervals over 30 group-shuffle re-splits for Layer A. The rule-based detector achieves recall at a high false-positive rate, while the two supervised ensembles achieve the highest AUPRC and F_1 .

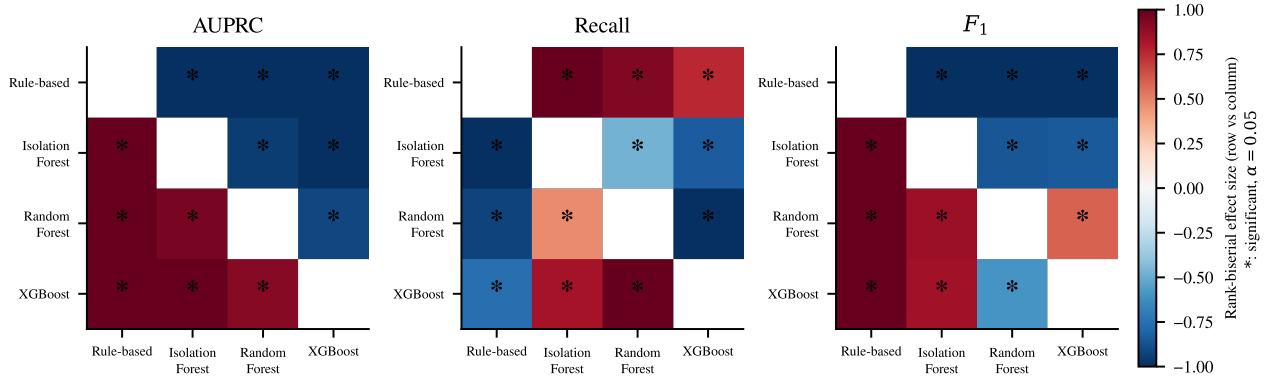


Fig. 5. Layer A Wilcoxon paired comparisons with Holm-Bonferroni correction. The cell color indicates the matched-pairs rank-biserial effect size of the row model relative to the column model, and asterisks indicate significance at $\alpha = 0.05$.

At Critical, 158 of 221 positives are found (recall = 0.715) and the 63 false negatives are all off-hours cases. At Medium level and higher, 13 false negatives are recovered and the recall level rises to 0.774. Consequently, the overall recall

deficiency is specific to one case and not spread out over the five cases. Working hours by themselves are unable to differentiate between off-hours anomalies and genuine cases where users are working in off-hours. It has to be used in

TABLE IX
LAYER B DETECTION RESULTS. VALUES ARE MEAN $\pm$ 95% CONFIDENCE INTERVAL REPORTED OVER 30 GROUP-SHUFFLE RE-SPLITS.

Model	Precision	Recall	F_1	AUROC	AUPRC	FPR
Rule-based	0.071 $\pm$ 0.009	0.705 $\pm$ 0.033	0.126 $\pm$ 0.013	0.680 $\pm$ 0.017	0.137 $\pm$ 0.008	0.375 $\pm$ 0.002
Isolation Forest	0.460 $\pm$ 0.066	0.485 $\pm$ 0.068	0.418 $\pm$ 0.033	0.851 $\pm$ 0.025	0.478 $\pm$ 0.034	0.032 $\pm$ 0.009
Random Forest	0.996 $\pm$ 0.003	0.669 $\pm$ 0.068	0.778 $\pm$ 0.066	0.924 $\pm$ 0.040	0.745 $\pm$ 0.060	0.000 $\pm$ 0.000
XGBoost	0.948 $\pm$ 0.054	0.671 $\pm$ 0.068	0.768 $\pm$ 0.069	0.918 $\pm$ 0.045	0.712 $\pm$ 0.059	0.003 $\pm$ 0.003

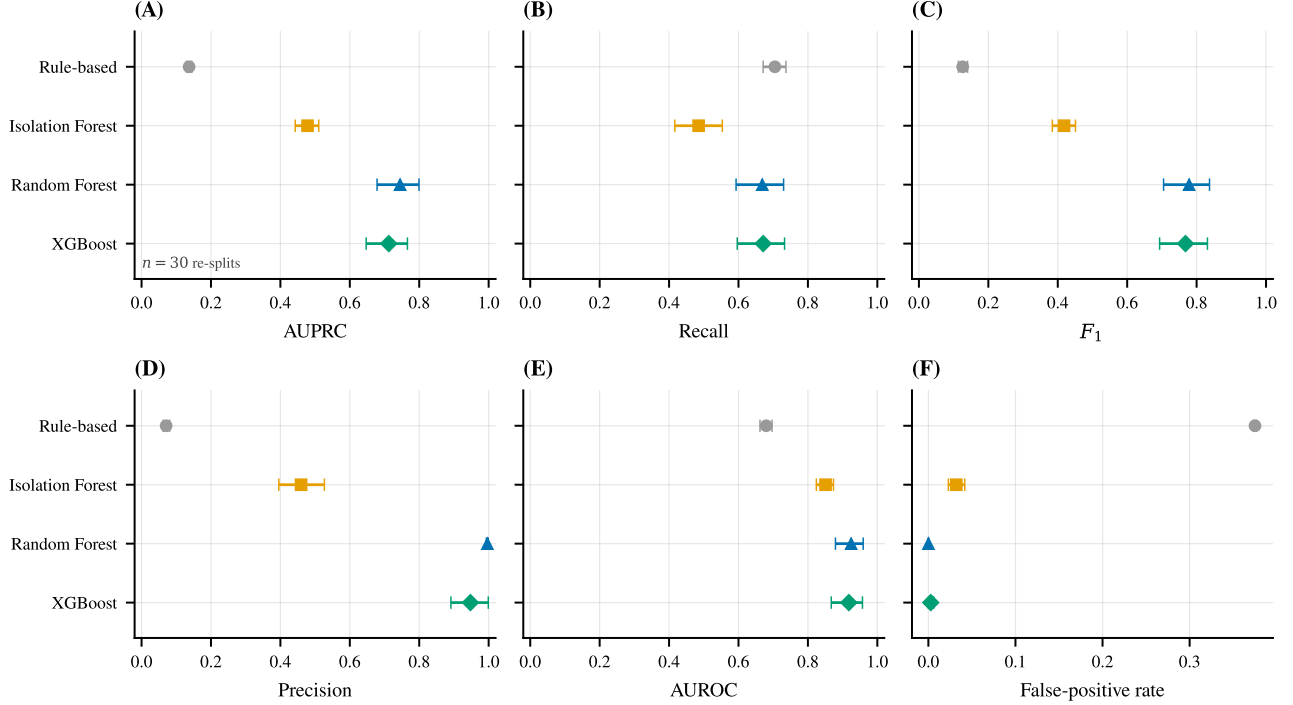


Fig. 6. Average detection metrics and 95% confidence intervals for Layer B. The large intervals for recall and AUPRC exemplify the sensitivity to which users and anomaly scenarios are included in a test set.

combination with other context like role–stage discrepancy or evidence sensitivity.

This is perhaps the most practically significant result from Layer B. All four of the other injected cases are identified with 100% accuracy at the Critical level, since each of them involves an anomaly combined with another piece of evidence — multiple failed login attempts plus an unusual device, or transition to the next stage plus mismatch between the role and stage (Table VII). Off-hours access, on the contrary, is a single weak anomaly by itself. A substantial share of legitimate investigators can work outside a [7, 21] window, which means that timestamp alone is not enough to distinguish between misuse and legitimate flexible hours.

4) *Feature noise sensitivity*: A feature-noise variant with $\sigma = 0.15$ is compared with the clean corpus with label flipping off. The AUPRC values for the rule-based approach decrease from 0.1466 to 0.1252 (−14.5%), the Isolation Forest from 0.5418 to 0.4727 (−12.7%), Random Forest from 0.8540 to 0.7727 (−9.5%), and XGBoost from 0.8229 to 0.6916 (−16.0%). It is sensitive to the change, but no system-

TABLE X
PER-SCENARIO DETECTABILITY ON THE LAYER B STRICT-TEMPORAL TEST PARTITION WITH RANDOM FOREST.

Scenario	Test n	Critical (%)	$\geq$ Medium (%)
Abnormal provenance rate	20	100.0	100.0
Credential misuse	58	100.0	100.0
Excessive downloads	20	100.0	100.0
Wrong-stage access	60	100.0	100.0
Off-hours access	63	0.0	20.6

wide failure. Random Forest provides the highest AUPRC. The greatest reduction for the supervised model occurs for XGBoost.

C. Layer B: Results from Integrated Pipeline

The results in this subsection come from a single integrated run rather than the 30 group-shuffle re-splits reported above, so the model the split protocol selects determines what the pipeline exercises. In the user-disjoint execution of Layer B, XGBoost yielded only three unique values for the validation

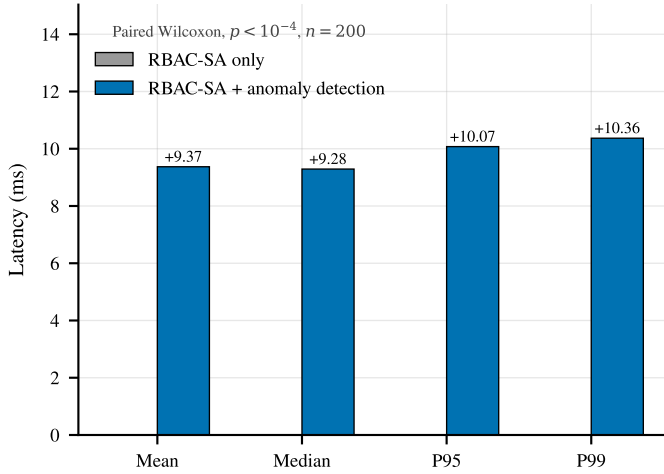


Fig. 7. Paired synchronous latency for RBAC-SA alone and RBAC-SA with anomaly detection ($n = 200$). The labels over the bars indicate the additional latency over the RBAC-SA baseline.

negatives, meaning that the three quantiles of false positives were almost equal, leading to an empty set for the High tier. Random Forest was selected by the strict-temporal approach; the distribution of its scores covered all four tiers, which utilized the entire path. The following values of latency, risk tier, XAI, blockchain, scalability, completion, and audit are calculated from this execution, and their divergence is mainly driven by the selected model.

1) *Latency of synchronous access*: Figure 7 represents latency of paired access using RBAC-SA alone compared to the latency of RBAC-SA plus anomaly score calculation in its entirety on the same 200 access attempts; it includes feature extraction, normalization, model inference, risk classification, and queuing up of the result, but not SHAP scoring and the blockchain operation itself, which happen asynchronously. Latency of the synchronous path was recorded using 200 paired accesses. The mean latency of the entire synchronous path is 9.371 ms (median 9.290 ms, P95 10.075 ms, P99 10.368 ms); the mean latency of RBAC-SA only is 0.00562 ms (P99 0.00763 ms). Wilcoxon signed-rank test: $W = 0$, $p = 1.44 \times 10^{-34}$; the path under examination was slower in every pair. Absolute overhead is the better indicator here — the selected Random Forest adds about 9.4 ms to the access-critical path while the user-disjoint XGBoost execution added about 2.2 ms. Thus, synchronous latency is model-dependent and not a property of the architecture.

2) *Stored explanations and overhead of XAI*: The execution of workers was timed on 20 marked samples from five different attack scenarios: average TreeSHAP computation took 5.32 ms, report rendering took 0.067 ms, while AES-GCM encryption and storage off-chain took 0.545 ms. That comes to a total of 5.93 ms. The average total time for workers is 69.15 ms; therefore, 63.22 ms are not accounted for by any of these three columns and represent the amount of time spent on deferred ledger interaction. The harness has no separate timing field for the chain link, so this allocation is an inference

based on that information and not a measured value. Therefore, the explanation calculation itself takes about 5.3 ms. The remaining 69 ms is the asynchronous audit process, still off the access-critical path.

Figure 8 contains example explanations calculated from encrypted off-chain reports. In the provided examples, past failures and hours of access are factors that increase the anomalousness, whereas the flags indicating stage transitions, type of request, and versioning of evidence lower the score. The examples above prove that the report that was stored can be decrypted and analyzed according to the canonical set of features. These are qualitative examples as there is no top- k corpus-level attribution stability measure used at this evaluation stage.

3) *Operational load and risk tiers*: Table XI and Fig. 9 illustrate the obtained operating point of validation based on the 7,500-event strict-temporal test partition (221 positives), including all four tiers. The Critical tier does a good job concentrating the signal: with only 2.3% access volume, it achieves 71.5% of cumulative recall with 92.4% tier precision. Expansion to Medium or above nearly triples the number of alerts (71.73 per 1,000) but with little increase in recall (77.4%). The Low tier (6,962 cases, 50 positives) is implicitly permitted but is never reached by the alert queue — that is a tier volume, not an alert volume.

Threshold sweep data is not provided here. In our current experiment setup, the harness recalculates thresholds based on the test partition and then scores the same, while the tier table uses thresholds derived from validation, so the threshold sweep would involve test-partition tuning.

4) *Overhead of Blockchain*: Table XII shows the outcome of 5,000 invocations of each of the functions in the Anomaly Ledger. The function `recordAnomalyDetected` utilizes an average of 230,431 gas and 292 bytes of calldata with a mean submit-to-confirm latency of 6.76 ms. On the other hand, the function `linkXAIReport` utilizes an average of 76,369 gas and 100 bytes of calldata with a mean submit-to-confirm latency of 6.20 ms. Thus, the total overhead of a fully linked anomaly is 306,800 mean gas and 392 bytes of calldata. Calldata total is a transaction-input footprint, not a contract-state growth measure. Gas is reported as deterministic EVM work, not monetary cost.

Since all these tests have been conducted on one node with automatic mining and without a consensus round, it means that the results illustrate the performance of the local JSON-RPC and EVM processing rather than confirmation latency in a multi-validator permissioned deployment. Moreover, the strict-temporal test was launched against a deeper local chain compared to the user-disjoint test; therefore, blockchain latencies for both tests will not be merged into one.

5) *Throughput and access path scalability for the blockchain*: As seen in Figure 10, the blockchain driver is able to cope with the load of 10 transactions/s and confirm all 300 of 300 transactions at 9.999 transactions/s. At 50 and 100 transactions/s targets, only 1,222 and 1,197 transactions are scheduled and submitted in the 30 s windows. However,

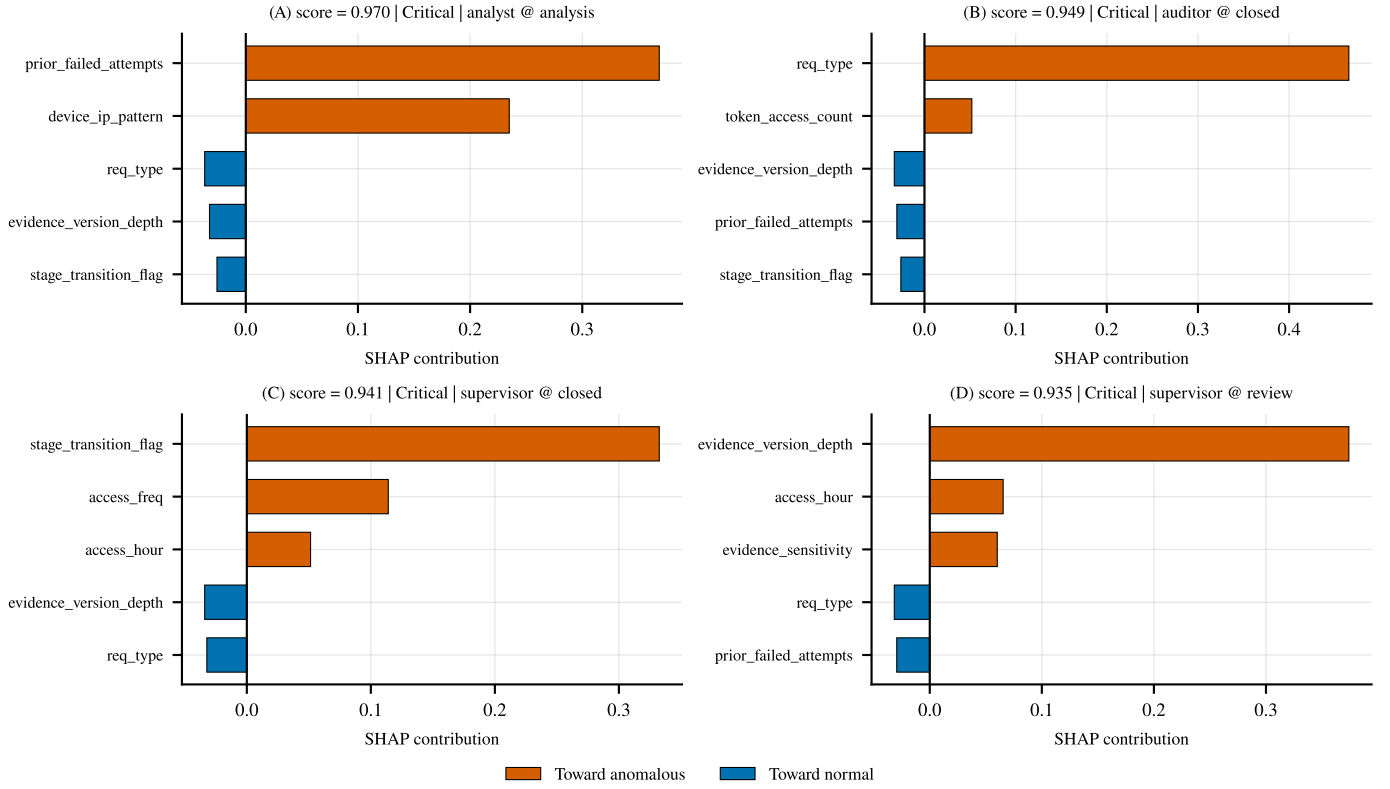


Fig. 8. The SHAP explanations were recovered by decrypting four of the stored off-chain reports, each selected such that each panel has a different leading contributor. Positive contributions pull the event toward the anomalous class and negative contributions pull it toward the normal class. The four events have distinct behavioral signatures: repeated failed attempts with an unknown device, request type with higher token access, a stage transition flag with increased access frequency and evidence version depth with off-hours timestamp, and occur at different roles and stages of investigation. The panels are qualitative examples, not a quantitative evaluation of attribution-consistency.

TABLE XI

STRICT-TEMPORAL RISK-TIER VOLUME AND OPERATIONAL LOAD IN LAYER B. LOW EVENTS ARE NOT ANALYST ALERTS AND ARE ALLOWED SILENTLY. THE LOW ROW IS A TIER VOLUME.

Tier	Volume	Positives	Tier precision	Cum. recall $\geq$ tier	Alerts/1,000 $\geq$ tier	Response
Low	6,962	50	0.0072	1.0000	1000.00	Allow
Medium	294	11	0.0374	0.7738	71.73	Alert supervisor
High	73	2	0.0274	0.7240	32.53	Step-up authentication
Critical	171	158	0.9240	0.7149	22.80	Step-up / policy block

TABLE XII

ANOMALY LEDGER OVERHEAD ON A LOCAL SINGLE-NODE HARDHAT NETWORK. GAS IS EVM WORK; LATENCY IS SUBMISSION TO MINED RECEIPT UNDER AUTOMINE.

Transaction	n	Mean gas	Min gas	Max gas	Calldata (B)	Mean (ms)	P99 (ms)
recordAnomalyDetected	5,000	230,431	215,445	235,417	292	6.76	8.97
linkXAIRReport	5,000	76,369	76,335	76,371	100	6.20	7.76

all transactions are validated, and yet the achieved throughput reaches its saturation point at 40.71 and 39.86 transactions/s, respectively, while the average latency rises to approximately 10.4 ms from 7.64 ms.

The saturation is a result of limitation in the submissions by the client side, and the driver fails to schedule an optimal number of 1,500 and 3,000 transactions in the windows.

Given that each anomaly link consists of two transactions, the client-side path indicates a rate of around 19.9 fully linked anomalies/s at the maximum rate reached. Tests run of this size repeatedly on the same host had been able to achieve rates ranging from 23 to 41 transactions/s, confirming that a 30 s single-client window measures the load generator and the host, not the ledger.

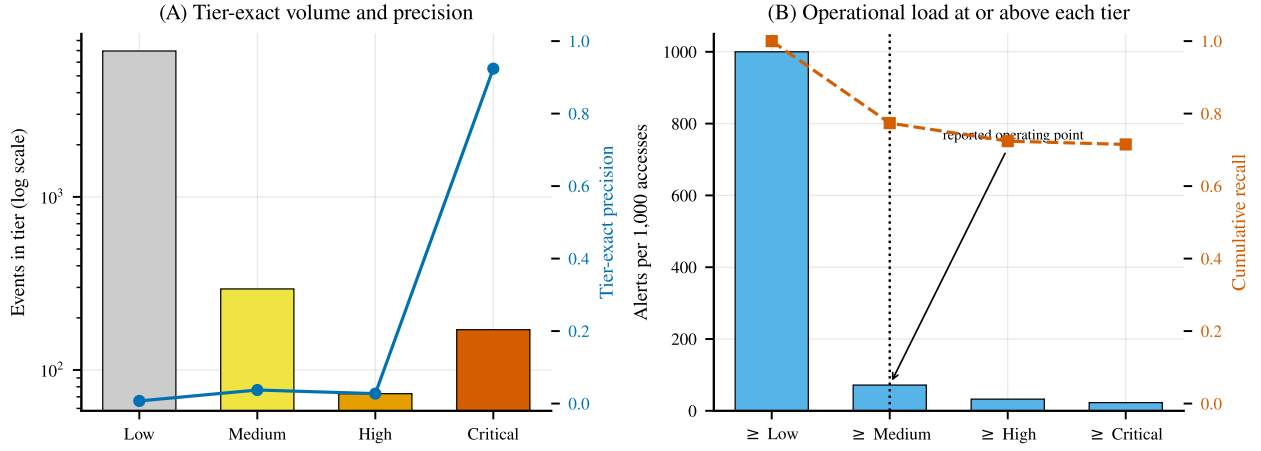


Fig. 9. Strict-temporal risk-tier behavior for Layer B. Panel (A) shows the tier-exact volume and precision, and panel (B) shows the cumulative recall and operational load at or above each tier. The dashed line indicates the Medium-or-higher reporting point.

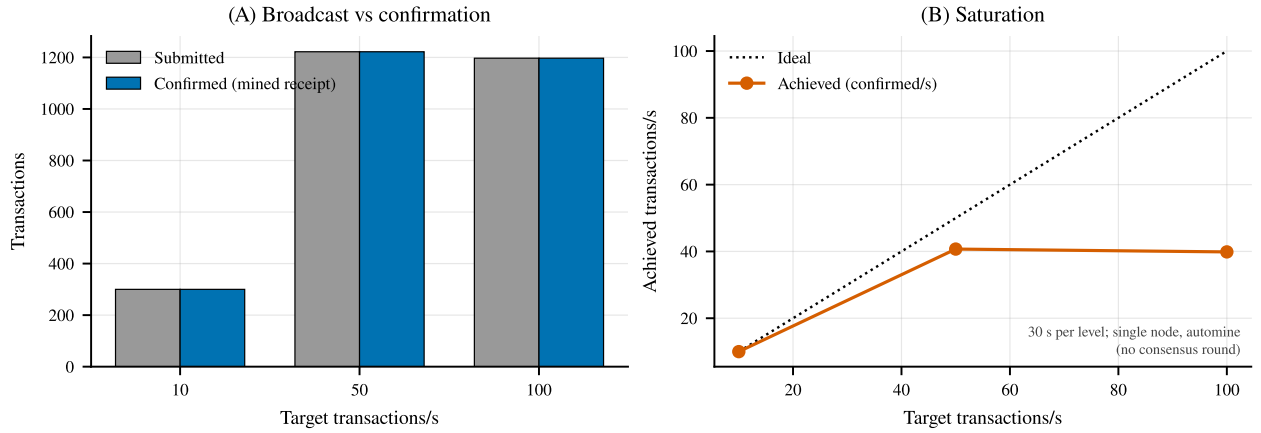


Fig. 10. Anomaly Ledger throughput under 30 s offered-load windows. Panel (A) shows transactions scheduled/submitted and confirmed; panel (B) shows the achieved confirmation rate. For higher targets, saturation is client-submission-bound on a single automining node.

The scalability of access paths is depicted in Figure 11. All requests succeed for all rates offered (10/10, 50/50, 100/100), with medians of latency equal to 14.42, 10.23, and 9.65 ms, while P99 values are 15.11, 13.31, and 16.98 ms. The baseline with RBAC-SA alone succeeds 100 out of 100 requests at the highest target, with average latency 0.028 ms; this means that almost the whole access-path cost is consumed by anomaly scoring rather than the permission check.

All the values shown are for the duration of one second in each rate level. Earlier runs of the same test completed only 50 out of 100 requests at the highest target. The metric thus evaluates short-term load management performance and not sustained throughput, and the values of the P99 metrics are derived from up to 100 observed tails. The multi-minute metric collection, independent runs, and warm-ups for measuring sustained loads will be explored in future work.

6) *End-to-end run completion integrity and audit*: In an end-to-end run, 25 anomalous requests which are allowed by RBAC-SA are submitted. All 25 of them are detected, put into the queue, processed by the background worker and logged

in $T_{\text{AnomalyDetected}}$ and linked in $T_{\text{XAIRReportLinked}}$. There is no drop in the number of events due to back pressure, no failure of the worker, the queue empties successfully, and the success ratio is 1.0. The run takes 1.874 s. This success rate is behavior observed under the stated parameters above and does not indicate durability: the in-memory queue remains non-durable until its first ledger transaction, and there has been neither an overflow nor any process failure for this run. Isolated queue benchmark takes 20 events in 1.317 s (15.19 events/s). The time interval between two ledger transactions for 50 events has a mean of 32.58 ms, a median of 31.81 ms, and a P99 of 45.95 ms, with none left unlinked.

Audit verification contains 65 anomaly entries with 40 from the operational test and 25 from the end-to-end test, which verify successfully without any failed hashes or unlinked records. For each verified entry, the model-artifact hash, decrypted report hash, policy version, and detection-to-link relationship are consistent with the artifacts recorded. These measurements confirm end-to-end integration under the stated conditions.

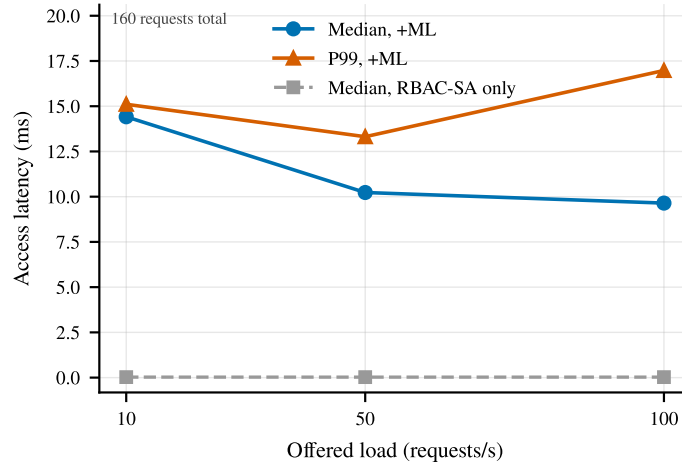


Fig. 11. Access-path latency for offered loads of 10, 50 and 100 requests/s. In this run both the Random-Forest path and the RBAC-SA-only baseline complete all offered requests. Each level is of the order of one second so that the percentiles are short-burst tail values rather than steady-state percentiles.

TABLE XIII

LAYER-B FEATURE-SET ABLATION. RANDOM FOREST, STRICT-TEMPORAL SPLIT, 7,500 TEST EVENTS WITH 221 POSITIVES. EACH ROW RETRAINS ON THE LISTED FEATURE GROUP ONLY.

Feature group	k	Prec.	Rec.	F_1	AUPRC	FPR
Full schema	15	0.988	0.715	0.829	0.783	0.0003
Forensic-specific	7	1.000	0.665	0.799	0.724	0.0000
Generic behavioral	8	0.279	0.575	0.375	0.341	0.0452
Policy-visible	5	0.050	0.434	0.089	0.137	0.2531

D. Ablation

The preceding results report the complete system. This subsection isolates what each part of it contributes, on the detection side and on the cost side.

1) *Feature-set ablation*: Table XIII retrains the selected Layer-B model on subsets of the canonical 15-feature schema, holding the corpus, the strict-temporal split protocol and the model family fixed. Because these are single-split fits rather than 30 re-splits, the full-schema row (AUPRC = 0.783) is a single draw from the interval reported in Table IX (0.745, 95% CI [0.679, 0.799]) and is not directly comparable to that mean; the arms are comparable to one another, which is what the ablation is for.

Three observations follow. First, the attributes RBAC-SA already holds at decision time — role, case stage, request type, evidence sensitivity and case identifier — carry almost no separating signal on their own (AUPRC = 0.137 at a false-positive rate of 0.253). A detector restricted to what the policy layer can already see is not operationally usable, which is the premise of the extension stated in Section II-C measured rather than asserted.

Second, the seven forensic-specific constructs are where the signal concentrates. Alone they reach AUPRC = 0.724 against 0.341 for the eight generic behavioral features that a CERT-style insider-threat detector would have access to — a factor of 2.1 — and they do so at a false-positive rate two

TABLE XIV

PER-REQUEST COST ATTRIBUTED TO EACH ADDED COMPONENT. ONLY THE SYNCHRONOUS ROW BLOCKS THE ACCESS DECISION.

Component	Path	Cost
RBAC-SA alone (baseline)	synchronous	0.006 ms
+ anomaly scoring	synchronous	9.371 ms
TreeSHAP attribution	asynchronous	5.32 ms
Report templating	asynchronous	0.07 ms
Encrypt and store	asynchronous	0.55 ms
Worker total per event	asynchronous	69.15 ms
$T_{\text{AnomalyDetected}}$	on-chain	230,431 gas
$T_{\text{XAIRReportLinked}}$	on-chain	76,369 gas
Detection to link interval	on-chain	32.58 ms

orders of magnitude lower. The generic set is what Layer A is restricted to, and the gap is consistent with Layer A's lower absolute scores.

Third, the two groups are complementary rather than redundant: the full schema improves on the better of the two subsets on AUPRC (0.783 versus 0.724) and recall (0.715 versus 0.665) at effectively unchanged precision. Neither group subsumes the other, which supports carrying the combined schema rather than the forensic subset alone.

2) *Cost ablation*: Table XIV decomposes the price of the additions by the path each one sits on. The distinction matters more than the totals: only the first row is paid inside the access decision.

Removing the explanation and ledger stages would return no time at all to the access path, because neither runs on it: the synchronous cost is 9.371 ms with or without them. What their removal would change is the audit record, not the response time. Conversely, the 9.365 ms that anomaly scoring adds to the RBAC-SA baseline is irreducible by moving work off the critical path — it is the scoring itself — and is the figure against which the detection gains in Table XIII should be weighed. Within the worker, TreeSHAP dominates the

attributable steps at 5.32 ms, though the measured per-event total of 69.15 ms is larger than their sum, the remainder being queue handling and ledger round-trips on the local network.

IX. CONCLUSION

This paper presents Anomaly-Aware ForensiBlock, an extension to stage-aware role-based access control, which introduces behavioral anomaly detection, risk-adaptive response, asynchronous explainability, and verifiable audit anchoring to authorized access for evidence. The architecture decouples the access-critical path from explanation generation and blockchain finalization. This preserves the existing forensic-provenance workflow with enhanced behavioral risk awareness.

The evaluation had two complementary objectives. On CERT r4.2 (Layer A), XGBoost achieved an AUPRC of 0.257 and an AUROC of 0.953 over 30 re-splits, whereas Random Forest yielded higher precision, higher F_1 , and a lower false-positive rate. Leakage-control analysis showed that recurrent users significantly boost supervised-model performance, implying user-disjoint evaluation is needed for behavioral generalization. Random Forest achieved an AUPRC of 0.745 and F_1 -score of 0.778 on the simulated full-schema corpus (Layer B), with four out of five injected misuse scenarios classified as Critical risk for all test events. Off-hours access remained the main limitation, with none of the 63 events in this category reaching the Critical tier.

In the integrated Random-Forest run, anomaly scoring added 9.37 ms mean synchronous overhead (10.37 ms P99). All 25 flagged requests were queued, processed, recorded, explained and linked. All 65 audited records were verified with no failed or unlinked entries. A fully linked anomaly required 306,800 mean gas units and 392 bytes of combined calldata on the local automining network. These results highlight that explainable anomaly-aware access control can be incorporated into a blockchain-based forensic framework with limited access-path overhead and verifiable end-to-end accountability within the scope of the artifact assessed.

A number of limitations apply to the study findings, which fall into three categories.

Dataset and model limitations. Layer A is a synthetic organizational benchmark rather than operational forensic-access data; Layer B depends on simulated access behavior, in which the wrong-stage-access scenario is bound deterministically to a single user across the whole corpus; and model selection was sensitive to timing variation around the 10 ms latency threshold.

Prototype and blockchain limitations. The artifact evaluated is a partial integration prototype — the access path runs in process rather than through the deployed ForensiBlock contracts, so the reported RBAC-SA baseline is an in-memory lookup and preservation of the provenance path is established by construction rather than by a five-contract experiment. The in-memory queue is non-durable until the first ledger transaction and has no retry, backoff, or dead-letter mechanism, so the ledger guarantees the integrity of recorded decisions

but not the completeness of anomaly capture. Reports are stored on a local filesystem rather than a managed object store, and encryption keys and pseudonymization salts have no rotation or **key management service (KMS) integration**. Contract assurance rests on targeted tests rather than fuzzing or formal verification.

Scalability limitations. All blockchain and access-path measurements were collected on a single-node Hardhat network with automatic mining and no consensus round, so they characterize local client and EVM execution rather than a multi-validator permissioned deployment; and the reported throughput and latency figures come from short (~ 30 s and ~ 1 s) offered-load windows rather than sustained-load testing.

Future work should therefore assess the full five-contract workflow, apply the framework to operational forensic-access logs, enhance the contextual modeling of off-hours activity, stabilize model selection, introduce durable asynchronous processing and evaluate deployment under a permissioned consensus network.

Anomaly-Aware ForensiBlock extends static forensic authorization to behavioral, explainable and auditable evidence governance without altering the underlying provenance mechanism.

REFERENCES

- [1] S. Bonomi, M. Casini, and C. Ciccotelli, “B-CoC: A Blockchain-Based Chain of Custody for Evidences Management in Digital Forensics,” in *International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2019)*, ser. Open Access Series in Informatics (OASISs), vol. 71. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 12:1–12:15.
- [2] I. Lavín and D. R. Llanos, “An analysis of blockchain solutions for digital evidence chain of custody,” in *Proceedings of the 7th International Congress on Blockchain and Applications (BLOCKCHAIN’25)*, Lille, France, 2025.
- [3] S. Ismail, M. Nouman, D. W. Dawoud, and H. Reza, “Towards a lightweight security framework using blockchain and machine learning,” *Blockchain: Research and Applications*, vol. 5, no. 1, p. 100174, 2024.
- [4] A. Jodeiri Akbarfam and H. Maleki, “Sok: Blockchain for provenance,” in *VLDB 2024 Workshop: Foundations and Applications of Blockchain*, 2024. [Online]. Available: <https://vldb.org/workshops/2024/proceedings/FAB/FAB-7.pdf>
- [5] O. S. Igonor, M. B. Amin, and S. Garg, “The application of blockchain technology in the field of digital forensics: A literature review,” *Blockchains*, vol. 3, no. 1, p. 5, 2025.
- [6] A. J. Akbarfam, M. Heidaripour, H. Maleki, G. Dorai, and G. Agrawal, “Forensiblock: A provenance-driven blockchain framework for data forensics and auditability,” in *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 2023, pp. 136–145.
- [7] J. Kim, M. Park, H. Kim, S. Cho, and P. Kang, “Insider threat detection based on user behavior modeling and anomaly detection algorithms,” *Applied Sciences*, vol. 9, no. 19, p. 4018, 2019.
- [8] D. C. Le and A. N. Zincir-Heywood, “Machine learning based insider threat modelling and detection,” in *2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*. Washington, DC, USA: IEEE, April 2019, pp. 1–6.
- [9] F. R. Alzaabi and A. Mehmood, “A review of recent advances, challenges, and opportunities in malicious insider threat detection using machine learning methods,” *IEEE Access*, vol. 12, pp. 30907–30927, 2024.
- [10] M. Wang, K. Zheng, Y. Yang, and X. Wang, “An explainable machine learning framework for intrusion detection systems,” *IEEE Access*, vol. 8, pp. 73 127–73 141, 2020.

- [11] S. Mane and D. Rao, "Explaining network intrusion detection system using explainable ai framework," *arXiv preprint arXiv:2103.07110*, 2021.
- [12] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron," *IEEE Access*, vol. 12, pp. 30 164–30 175, 2024.
- [13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.
- [14] B. Lindauer, J. Glasser, M. Rosen, and K. Wallnau, "Generating test data for insider threat detectors," *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, vol. 5, no. 2, pp. 80–94, 2014.
- [15] D. C. Le, A. N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 30–44, March 2020.
- [16] H. K. Ofori, K. Bell-Dzide, W. L. Brown-Acquaye, F. Lempogo, S. O. Frimpong, I. E. Agbehadj, and R. C. Millham, "Application of machine learning and deep learning techniques for enhanced insider threat detection in cybersecurity: A bibliometric review," *Symmetry*, vol. 17, no. 10, p. 1704, 2025.
- [17] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422.
- [18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [20] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [21] A. M. Salih, Z. Raisi-Estabragh, I. Boscolo Galazzo, P. Radeva, S. E. Petersen, K. Lekadir, and G. Menegaz, "A perspective on explainable artificial intelligence methods: SHAP and LIME," *Advanced Intelligent Systems*, vol. 7, no. 1, p. 2400304, 2025.
- [22] M. T. Ribeiro, S. Singh, and C. Guestrin, "“Why Should I Trust You?” Explaining the Predictions of Any Classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144.
- [23] L. S. Shapley, "A value for n -person games," in *Contributions to the Theory of Games, Volume II*, ser. Annals of Mathematics Studies, H. W. Kuhn and A. W. Tucker, Eds. Princeton University Press, 1953, no. 28, pp. 307–317.
- [24] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [25] N. Xiao, Z. Wang, X. Sun, and J. Miao, "A novel blockchain-based digital forensics framework for preserving evidence and enabling investigation in industrial Internet of Things," *Alexandria Engineering Journal*, vol. 86, pp. 631–643, 2024.
- [26] N. Kumar and R. Ali, "Smart-FIoT: A smart contract based efficient authentication framework for forensics IoT," *Cluster Computing*, vol. 28, p. 895, 2025.
- [27] S. Nath, K. Summers, J. Baek, and G.-J. Ahn, "Digital evidence chain of custody: Navigating new realities of digital forensics," in *2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 2024, pp. 11–20.
- [28] L. Loffi, G. L. Camillo, C. A. De Souza, C. M. Westphall, and C. B. Westphall, "Management of the chain of custody of digital evidence using blockchain and self-sovereign identities: A systematic literature review," *IEEE Access*, vol. 13, pp. 77 804–77 832, 2025.
- [29] I. A. Haji, S. D. Mohammed, and K. M. Mousa, "Blockchain based chain of custody and digital evidence legality in post-conflict prosecutions," *Frontiers in Blockchain*, vol. 9, p. 1801364, 2026.
- [30] K. Fei, J. Zhou, Y. Zhou, X. Gu, H. Fan, B. Li, W. Wang, and Y. Chen, "LaAeb: A comprehensive log-text analysis based approach for insider threat detection," *Computers & Security*, vol. 148, p. 104126, 2025.
- [31] Z. Wei, U. Rauf, and F. Mohsen, "E-Watcher: Insider threat monitoring and detection for enhanced security," *Annals of Telecommunications*, vol. 79, pp. 819–831, 2024.
- [32] S. A.-D. Qawasmeh and A. A. S. AlQahtani, "Beyond firewall: Leveraging machine learning for real-time insider threats identification and user profiling," *Future Internet*, vol. 17, no. 2, p. 93, 2025.
- [33] O. Arreche, T. R. Guntur, J. W. Roberts, and M. Abdallah, "E-XAI: Evaluating black-box explainable AI frameworks for network intrusion detection," *IEEE Access*, vol. 12, pp. 23 954–23 988, 2024.
- [34] M. T. Islam, M. K. Syfullah, M. G. Rashed, and D. Das, "Bridging the gap: Advancing the transparency and trustworthiness of network intrusion detection with explainable AI," *International Journal of Machine Learning and Cybernetics*, vol. 15, no. 11, pp. 5337–5360, 2024.
- [35] N. Khan, K. Ahmad, A. Al Tamimi, M. M. Alani, A. Bermak, and I. Khalil, "Explainable AI-based intrusion detection systems for Industry 5.0 and adversarial XAI: A systematic review," *Information*, vol. 16, no. 12, p. 1036, 2025.
- [36] D. Goyal, S. K. Singh, S. Kumar, V. Chilkoti, K. T. Chui, C.-H. Hsu, and B. B. Gupta, "An explainable AI framework for intrusion detection using SHAP and LIME," in *2025 IEEE 14th Global Conference on Consumer Electronics (GCCE)*. IEEE, 2025, pp. 1315–1319.
- [37] Z. Khalid, F. Iqbal, and M. Saqib, "Bridging knowledge gaps in digital forensics using unsupervised explainable AI," *Forensic Science International: Digital Investigation*, vol. 53, p. 301924, 2025.
- [38] M. C. Calzarossa, P. Giudici, and R. Zieni, "An assessment framework for explainable AI with applications to cybersecurity," *Artificial Intelligence Review*, vol. 58, p. 150, 2025.
- [39] M. N. Nobi, M. Gupta, L. Praharaj, M. Abdelsalam, R. Krishnan, and R. Sandhu, "Machine learning in access control: A taxonomy and survey," *arXiv preprint arXiv:2207.01739*, 2022.
- [40] G. Hasel Mehri, C. Morisset, and N. Zannone, "Towards explainable access control: [BlueSky paper]," in *Proceedings of the 30th ACM Symposium on Access Control Models and Technologies*. ACM, 2025, pp. 117–126.